Assignment

Report(stage 1,2,3)

Chinese Name:	Liyan Wang
ESSEX Number:	2107394
Registration Number:	2021117541
Model Code:	CE264-NWU

Apr.2023

NWU&ESSEX

catalogue

Stage1 Clock Divider and 7-segment display driver
1. Introduction 1
2. Circuit Designs 1
2.1 Clock Divider 1
2.2 7-segment display driver 2
3 Testing 4
3.1 The testing of Clock Divider 4
3.2 The testing of 7-segment display driver 5
Stage2 VGA driver
1 Introduction 6
2 Circuit Designs 6
2.1 VGA timing circuit 6
2.1.1 General introduction 6
2.1.2 Functional description of Horizontal and Vertical timing 9
2.2 RGB circuit 12
3 Testing 13
3.1 SYNC/BLANKING testing 13
3.2 RGB and VCOUNT/HCOUNT testing 16
Stage3 Dynamic Programmable Video Test Card
1. Introduction 17
2. Circuit Design 17
2.1 Colored rectangle Design 18
2.2 Moving rectangle Design 20
2.2.1 General introduction 20
2.2.2 Functional and logical description of individual circuit 21
2.3 Coordinate recording Design 25
2.3.1 Horizontal coordinate recording 26
2.3.2 Vertical coordinate recording 27
2.4 2 colors storage and 2 coloring modes Design 27
2.4.1 1-bit register 28
2.4.2 Single-colour storage 30
2.4.3 Mode selection circuit 31
3. Testing 33
3.1 Colored rectangle 33
3.2 Moving rectangle and Coordinate recording 33
3.3 2 colors storage and 2 coloring modes Design 34
Presentation, discussion, self-reflective statement
1 Discussion of hierarchy 35
2 self-reflection 36
2.1 learning 36
2.2 Debugging and Challenges 36
3 References and background reading work 37
Appendix

Stage1 Clock Divider and 7-segment display driver

1. Introduction

In this stage, there are two parts that I need to complete. The former is the Clock Divider and the later is the 7-segment display driver. For the first part, I created and tested a Clock Divider with one input and three outputs. Input is a 100MHz clock signal from the Basys3 board; the three outputs are 25MHz, 500Hz and 0.5Hz clock signals, which will be used frequently in subsequent experiments. For the second part, I designed and tested a 7-segment display driver with 16 switch inputs and 4 BCD digits outputs which were displayed by 7 LED. The 16 switches will be divided into four groups of four to control each of the 4 BCD digits. Each digit displayed on the four-digit seven-segment display is a binary number consisting of the four switches of the group in which it is located, ranging from 0-9 (0000 to 1001).

2. Circuit Designs

2.1 Clock Divider

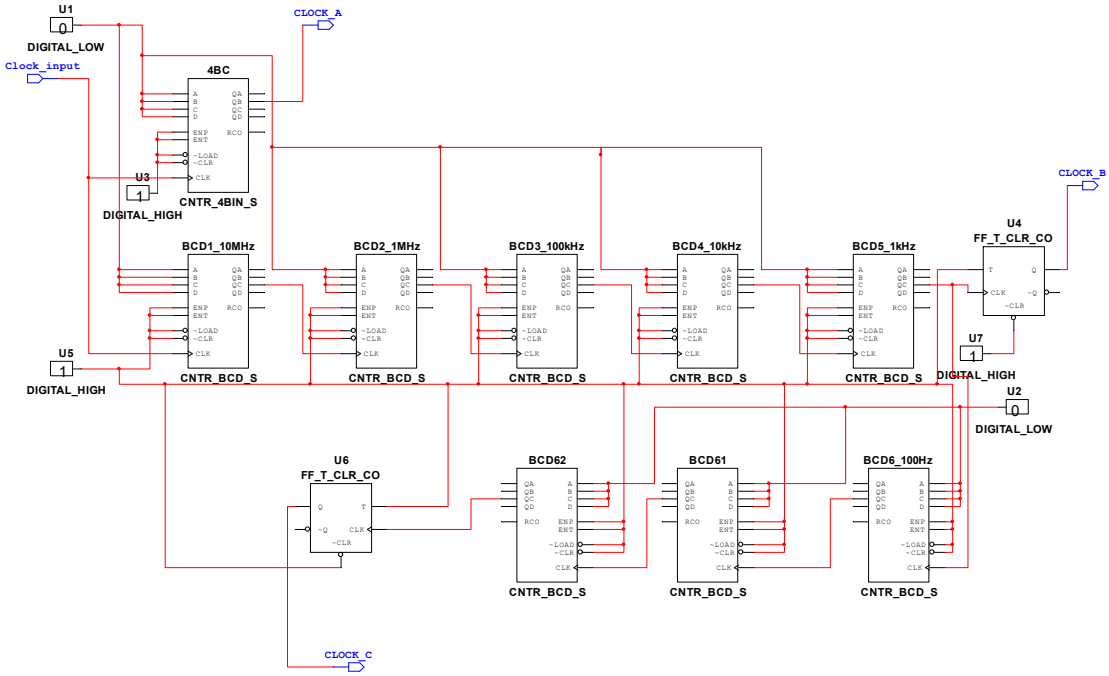
The following schematic shows my Clock Divider. This Clock Divider consists of 2 T flip-flops, one 4 Bit positive-edge-triggered Synchronous Counter and 8 BCD Decade positive-edge-triggered Synchronous Counter, with one clock input and three clock output.

For the first output, 100 MHz needs to be converted to 25 MHz. In this circuit, I use one 4 Bit positive-edge triggered Synchronous Counter to achieve it. As for this counter, when the ENP, ENT and LOAD inputs are high, the count mode will be enabled; meanwhile, for we don't need to reset the counter, I also keep the CLR input high. I set the parallel data inputs (A, B, C, D) to (0000), so that it can begin to count from 0. By the characteristics of binary counting we can see that LSB (QA) has the highest frequency and MSB (QD) has the lowest frequency. From QA to QD, the frequency decreases to twice the former in order. Since the counter counts once on each rising edge, i.e. once per cycle, and QA requires two counts (0,1) for one cycle, the frequency of QA is half the frequency of the input clock. By analogy, the frequency of QB is one quarter of the input frequency, QC is one eighth, etc. Therefore, We need to output 25MHz at CLOCK_A, a quarter of the input clock, so just connect the output CLOCK_A at QB.

For the second output, we need to get 500Hz output. Obviously, the exact 500Hz cannot be obtained directly by reducing the frequency by a factor of two in sequence, as was done with the first output, and is also less efficient, so it is necessary to choose other counters. Here BCD Decade positive-edge-triggered Synchronous Counter is a good choice. Since this counter can only count from 0 to 9 in decimal, i.e., it clears to zero as soon as it reaches 1001, the frequency of each output bit is different compared to the previous counter. After setting up the initial conditions modelled on the previous counter, we can see that: For QA, it is still a simple 0 and 1 cycle, so its frequency is still half of the input frequency. For QB, the cycle becomes 0011001100, QC' s cycle becomes 0000111100, and QD' s cycle becomes 0000000011, so all three

count 10 numbers in one cycle at one-tenth the input frequency. Therefore, I chose to use 5 of these counters to reduce the frequency from 100MHz to 1kHz, and because the duty cycle of the QC is overall better, I chose to use the QC as the output of each stage. In order to change 1kHz to 500Hz, I have used a T-flip-flop. For T-flip-flops, set CLR and T high, so that it will toggle the output toggle on the rising edge of each clock. Therefore, connecting 1kHz to the clock position of the T-flip-flop gives a 500Hz output.

For the third output, we need to get 0.5Hz output. Obviously, 0.5 is one thousandth of 500, so the corresponding 0.5Hz can be obtained along the lines of CLOCK_B by using 1 T-flip-flops and 3 BCD Decade positive-edge-triggered Synchronous Counter and connecting the inputs to 1kHz of the last output.



P1 the design of Clock Divider

2.2 7-segment display driver

The following schematic shows my 7-segment display driver, which consists of 4 MUX4_1, one 4 Bit positive-edge-triggered Synchronous Counter, one DEC_BCD_7, one 2 to 4 Line Decoder and our own Clock Divider from last section. On top of that, we have 16 switch inputs and 1 clock input which are directly connected to the 100MHz clock and 16 switches (SW0-SW15) on the Basy3 board; meanwhile, we have 4 outputs (AN0-AN3) to control 4 BCD digits and 7outputs (CA-CG) to control 7 LED for each digit.

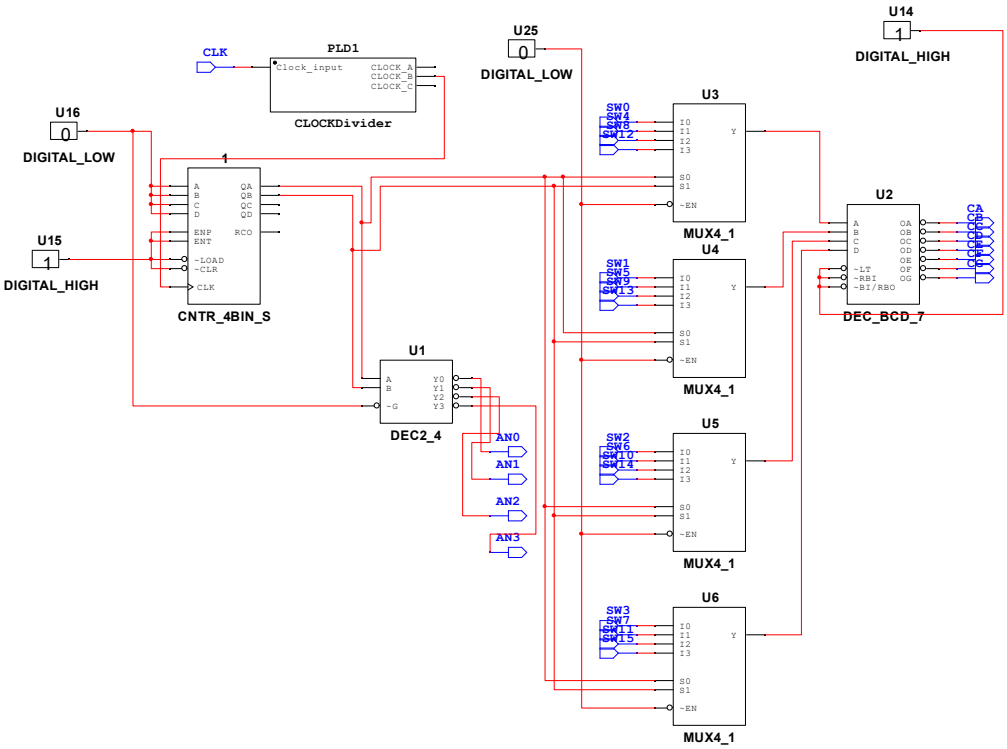
In this circuit I have divided the 16 switches into 4 groups of 4 each to control 4 BCD digits. Four switches in each group are used to represent just the 4-digit binary form of the BCD digit. The details of the grouping are as follows, SW0-SW3 control the number corresponding to AN0, similarly SW4-SW7 control AN1, SW8-SW11 control AN2, SW12-SW15 control AN3.

I made a 2-bit binary counter and used 500Hz as the drive clock so that it constantly varies from 00-11. Connecting a 4 Bit positive-edge-triggered Synchronous

Counter modelled on the wiring of the previous circuit, and also connecting the clock to the CLOCK Divider's 500Hz clock, you get a 4-bit binary digit counter, and since we only have four BCD digits, we only need to use the last two digits of its output (QA, QB). Subsequently, connect the two output pins to the two-digit decoder and connect each of the four outputs of the decoder to the pins corresponding to the four BCD digits (AN0-AN3). The two-digit decoder selects Y0, Y1, Y2, and Y3 to give a high level according to the binary number (A is LSB) 00,01,10,11 formed by inputs A and B, respectively. Therefore, we have achieved the repeated counting of 00-11 at 500Hz, so the circuit will repeatedly select the four BCD digits in a short period of time, and due to its fast frequency and our eyes have the characteristic of visual transience, the four BCD digits will be presented in the four-digit seven-segment display at the same time.

I then added the switch in with four MUX4_1. For MUX4_1, it will select the inputs to which I0, I1, I2, and I3 are connected for output based on the binary numbers (S1 is LSB) consisting of S1 and S2: 00,01,10,11, respectively, when the enable terminal goes high. Therefore, I let the first MUX4_1 connects the first bit of the binary form of each BCD digit, i.e. SW0, SW4, SW8, SW12; the second MUX4_1 connects the second bit, i.e. SW1, SW5, SW9, SW13; and so on. Meanwhile, I connected the S0 and S1 pins of the MUX4_1 to the QA and QB of the upper two-digit binary counters, so that while the decoder selects each BCD digit, the four MUX4_1s likewise select the four switches in the group in which that digit is located.

Since DEC_BCD_7 can directly realize the conversion between BCD digits to 7-Segment Driver, we can directly connect the outputs of 4 MUX4_1 to its inputs and connect the DEC_BCD_7 outputs to the CA-CG corresponding to the 7 LED of each digit to complete the design of this experiment.



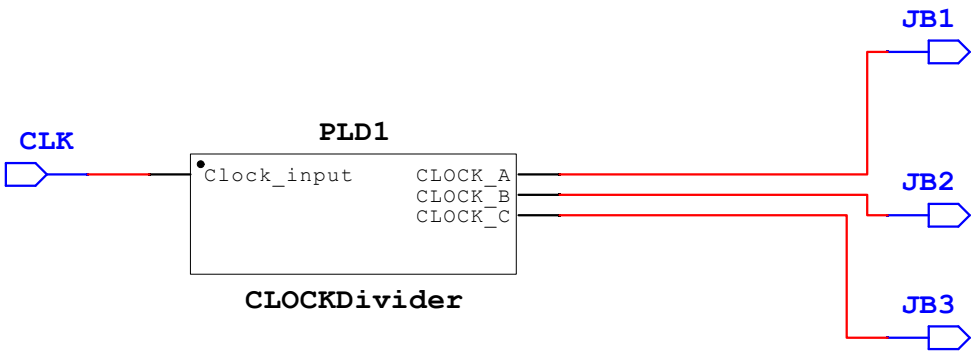
P2 the design of multiplexing circuit

3 Testing

3.1 The testing of Clock Divider

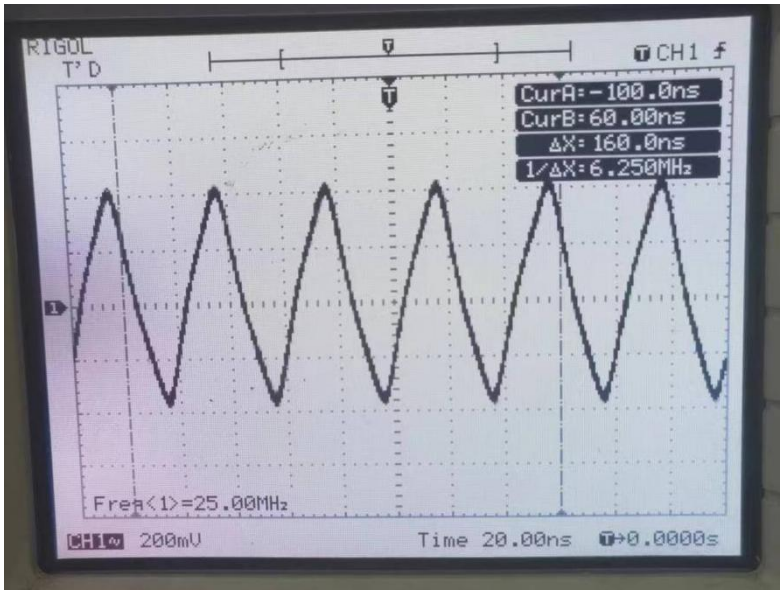
For the Clock Divider, we need to test three output frequencies: 25MHz, 500Hz, and 0.5Hz. It is easy to use oscilloscope to test the 25MHz and 500Hz, but due to the 0.5Hz is too slow for oscilloscope, we test it with a LED video.

To make it easier for the oscilloscope to measure the output waveform, I built a test circuit by connecting each of the three outputs to the output pins JB1, JB2, and JB3 on the Basys3 board.Subsequently, by connecting the pins of an oscilloscope to the Basys3 board, I can test the corresponding output waveform. The test circuit is shown below.



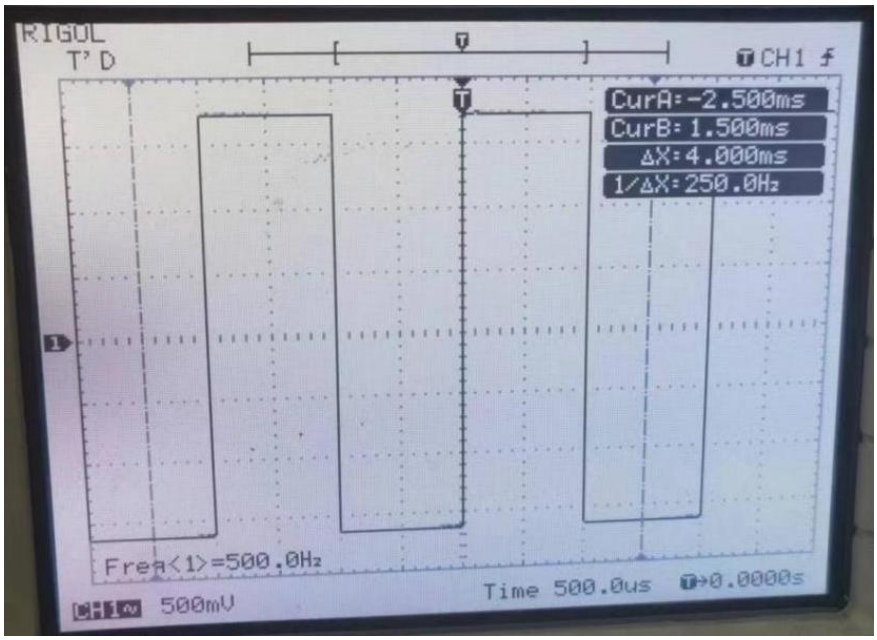
P3 The testing circuit(oscilloscope)

For the first output, the waveform presented by the oscilloscope are as follows. We can clearly find that the frequency is equal to 25MHz, and the waveform is similar to sin wave.



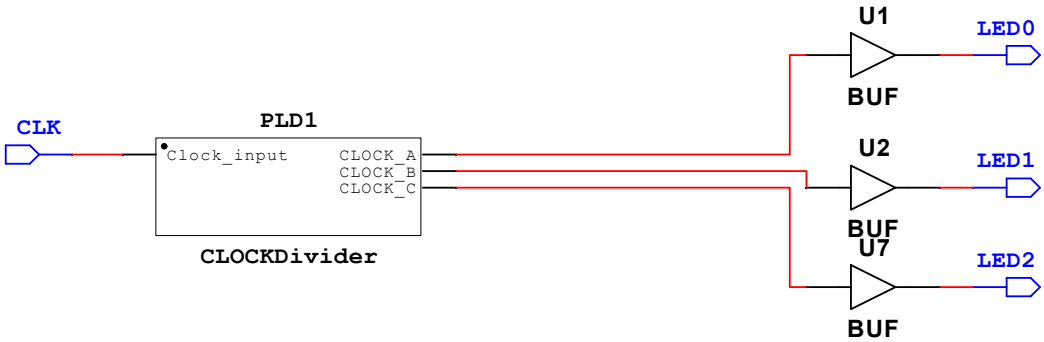
P4 the waveform of 25MHz

For the second output, the waveform presented by the oscilloscope are as follows. We can clearly find that the frequency is equal to 500Hz, and the waveform is similar to square wave.



P5 the waveform of 500Hz

For the third output, I built a test circuit and connected the output to the LED with a Buffer on the Basys3 board, so that we can get a more stable LED flicker and determine the correctness of the 0.5Hz by the blinking interval of the LED. The testing video is Stage1_0.5Hz_testing. The test circuit is shown below.



P6 The testing circuit(LED)

Through the video we found that LED2 keeps flashing in a cycle of about 2 seconds, which is in line with the 0.5Hz frequency, so the experiment is more successful.

3.2 The testing of 7-segment display driver

For the testing of 7-segment display driver, we can judge the correctness of the Basys3 board directly from its experimental results. The testing video is Stage1_7-segment display driver_testing.

Through the video we found that SW0-SW3 control the number corresponding to AN0, SW4-SW7 control the number corresponding to AN1, SW8-SW11 control the number corresponding to AN2, SW12-SW15 control the number corresponding to AN3, and the binary number corresponds to the decimal number correctly. At the same time display will only show the number 0-9, when the input is greater than 1010, it will not be displayed.

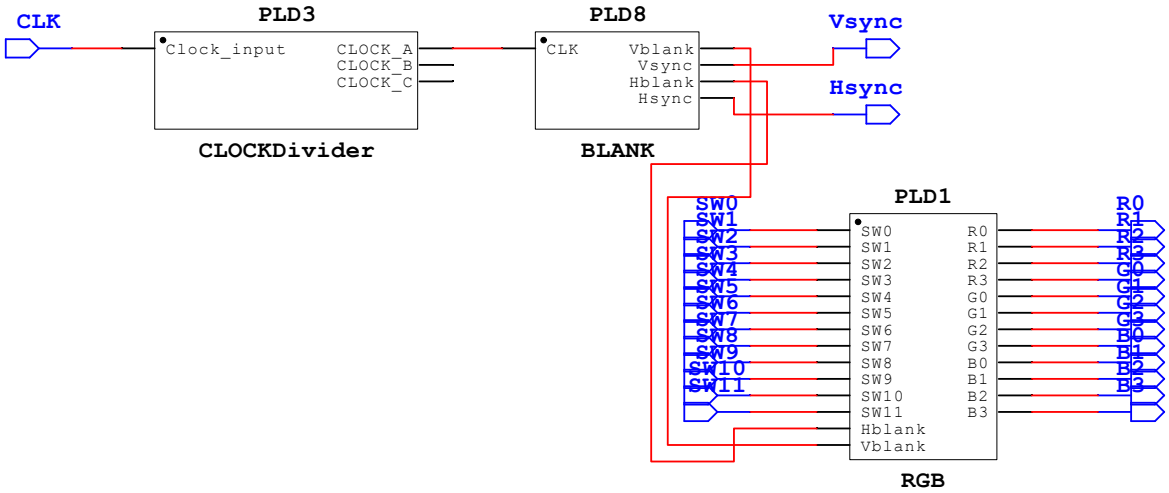
Stage2 VGA driver

1 Introduction

In this stage, I need to complete a design of VGA driver. Specifically, I'll be using a 640 x 480 raster and a 25MHz pixel clock from stage1's Clock Divider with my VGA timing circuit and RGB circuit to make the whole screen change to the colour I want.

2 Circuit Designs

The following schematic shows my VGA driver. I use a hierarchical structure to divide the circuit into three main sub-systems. The first one is the Clock Divider from stage1, so the essay won't expand on it here; the second is the VGA timing circuit, I called it BLANK, which is the main circuit of VGA driver with a input of clock and four outputs which represent VBlank, HBlank, VSync, HSync; the third is the RGB circuit, controlling the colour of screen. Next I'll focus on the driving circuit and RGB circuit with 12 switch inputs, 2 'blank' signal input and 12 outputs correspond exactly to the 12 sections of the RGB colour(R0-R3,G0-G3,B0-B3).



P7 The design of VGA driver

2.1 VGA timing circuit

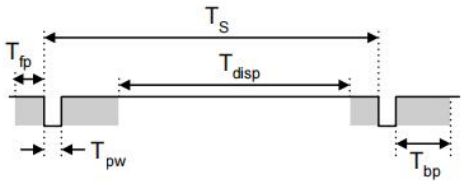
2.1.1 General introduction

Overall, the VGA timing circuit can be composed of two parts: Horizontal timing and Vertical timing.

For the Horizontal timing, it can also be divided into three parts: HSync, HBlank, HCOUNT. The HSync signal indicates when the monitor should start a new scan line. It is a low-level pulse that appears at the beginning of each horizontal scan line.The HSync signal tells the monitor when to start drawing the next line of pixels and to synchronize the image horizontally. HBlank indicates that the display starts or stops drawing pixel data during horizontal scanning and prepares for work. The HCOUNT is a counter used to calculate the position of the horizontal scan line. It is incremented during the horizontal scan and reset at the end of each horizontal scan line.

Similarly, for the Vertical timing, it can be composed by VSync, VBlank, VCOUNT. The VSync signal indicates when the monitor should start a new frame. It is a low-level pulse that appears at the beginning of each frame.The VSync signal tells the monitor when to start drawing the next frame and synchronizing the image vertically. VBlank indicates that the display starts or stops plotting pixel data and prepares for work. The VCOUNT is a counter used to calculate the position of a frame. It is incremented during the vertical scan and reset at the end of each frame.

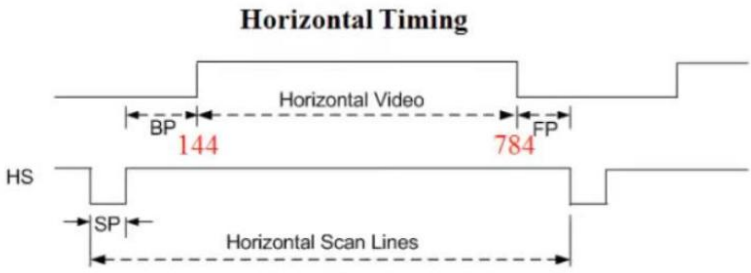
On top of that, The basic idea of VGA timing circuit is to output the correct HSync, VSync, HBlank, VBlank signals depending on the parameters of the screen in hand (640 x 480). The relevant parameters of the scanning and blanking signals can be obtained by referring to the manuals.



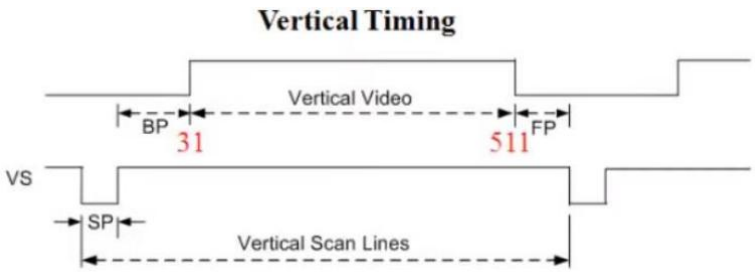
Symbol	Parameter	Vertical Sync			Horiz. Sync	
		Time	Clocks	Lines	Time	Clks
T_S	Sync pulse	16.7ms	416,800	521	32 us	800
T_{disp}	Display time	15.36ms	384,000	480	25.6 us	640
T_{pw}	Pulse width	64 us	1,600	2	3.84 us	96
T_{fp}	Front porch	320 us	8,000	10	640 ns	16
T_{bp}	Back porch	928 us	23,200	29	1.92 us	48

P8 The relevant parameters

Therefore, according to the above data, it is easy to get the waveform of four relevant signals. The former is horizontal timing and the later is the vertical timing.



P9 Horizontal timing



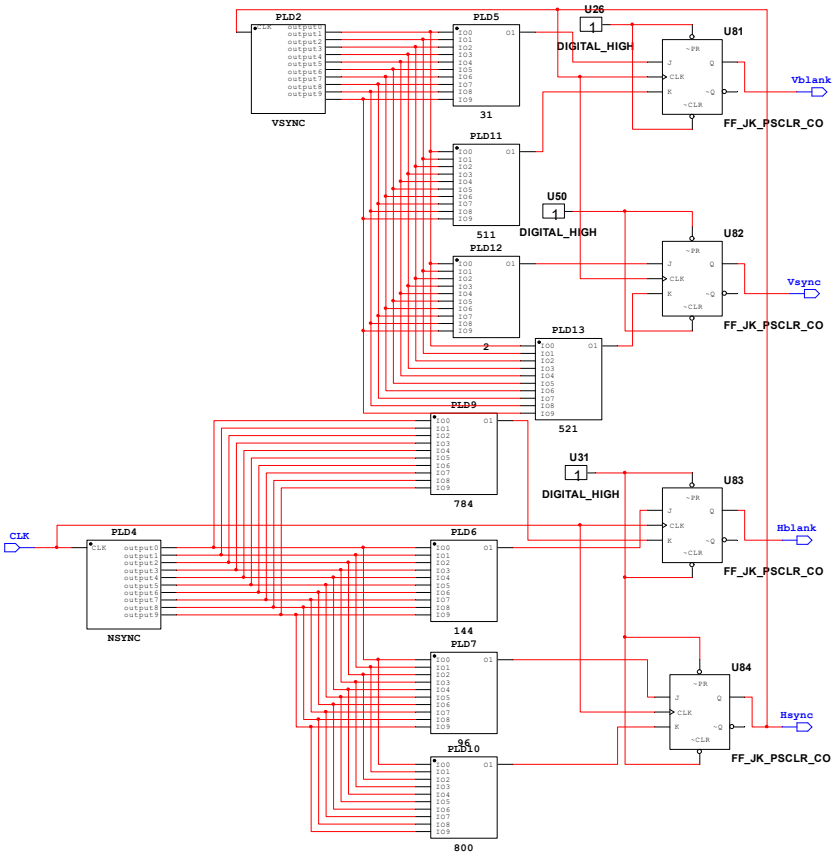
P10 Vertical timing

Specifically, horizontal timing has two square signals, which both have 800 pixels

with the same frequency of 25MHz. The first one is HBlank signal, which has three low level signals and one high level signal. The three low level signals are pulse width, back porch, front porch; the high level signals is display time. Based on the above reference data, the characteristics about this square wave signal can be simply obtained: the frequency of this signal is 25 MHz, the single cycle is divided into 800 pixels, and it is high between pixel 144 and pixel 784, and low for the rest. Meanwhile, the second one is HSync signal, which is made by changing the back porch and front porch from low to high on the basis of the HBlank signal. Similarly, it has the following characteristics: the frequency of this signal is 25 MHz, the single cycle is divided into 800 pixels, and it is high between pixel 96 and pixel 800, and low for the rest.

For the same reason, the characteristics of VBlank and VSync signals can be profiled: as to the VBlank signal, its' cycle is divided into 521 pixels, and it is high between pixel 31 and pixel 511, and low for the rest; as to the VSync signal, its' cycle is divided into 521 pixels, and it is high between pixel 2 and pixel 521, and low for the rest.

The following schematic shows my design of VGA timing circuit. It is clear that it also consists of many sub-systems. Overall, the circuit has 10 subsystems, which can be divided into two categories according to horizontal and vertical. Since these two categories serve horizontal timing and vertical timing respectively, which have the same principle and similar functions, horizontal timing is introduced here in detail, while vertical timing is not specified.



P11 The circuit of VGA timing

2.1.2 Functional description of Horizontal and Vertical timing

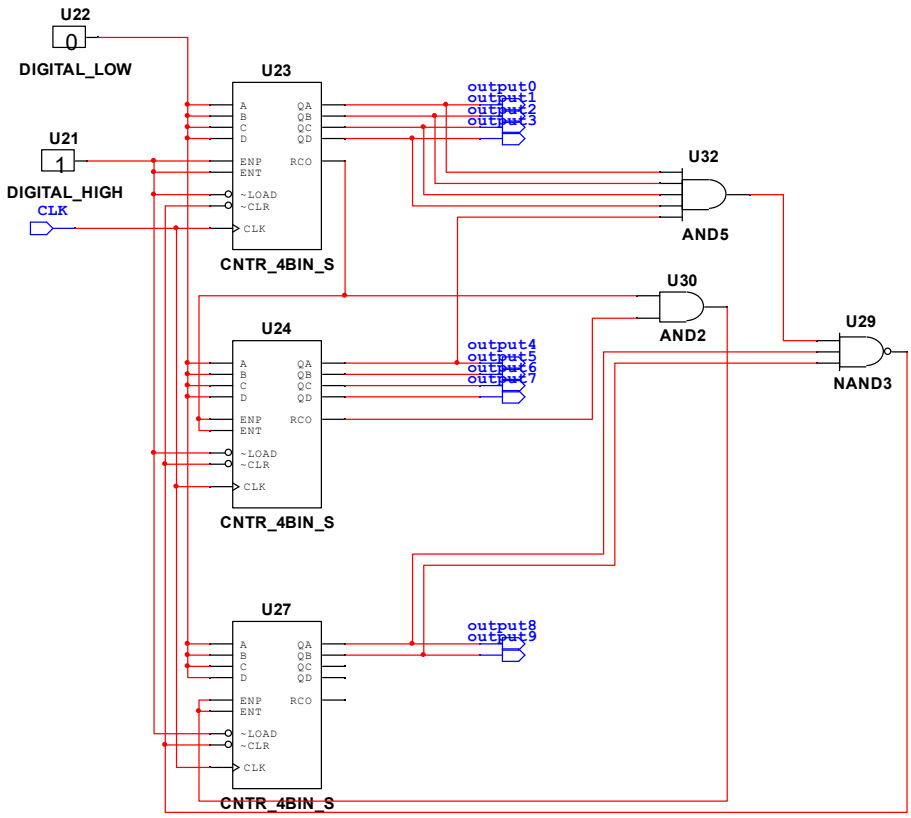
As to the horizontal timing, we have five sub-systems called HSYNC, 144, 784, 96, 800 as well as two JK flip-flop. Meanwhile, it has a clock input and two output called HBLANK and Hsync separately, which will output the signals we need.

The following schematic shows the design of HSYNC. Just because it has this name doesn't mean it outputs the signal as HSync. It is a ten-digit synchronized binary counter capable of counting a total of 800 numbers. Since it starts counting from 0, it has a counting range of 0-799 and then loops continuously. This circuit consists of four 4 Bit positive-edge-triggered Synchronous Counter and a number of and gates and and non-gates, the inputs are clock inputs and the outputs are in the form of a ten-bit binary from 0-799 (where output0 is the LSB and output9 is the MSB). The use of these four counters here is similar to Stage1 in that they have a counting function when ENP, ENT and LOAD are all high and count from 0(DEC) based on input 0000. The difference is that since this is a cascade of three four-bit binary counters, there are inter-counter rounding issues and start-up timing issues for the different counters.

Specifically, for the counting of the lowest four counters, can be modelled on Stage1, directly connected to the high level in the ENP and ENT terminals to make it work, however, for the latter two counters, in the process of counting from 0000 to 1111 should have been in a dormant state, until the number of digits counted is greater than 1111, the second counter will not begin to work; similarly, until the number of digits counted is greater than 1111111111, the third counter will not begin to work. Luckily, the 4 Bit positive-edge-triggered Synchronous Counter has a feed function, i.e., when QA, QB, QC, and QD are all 1, the RCO will output a high level. Therefore, the RCO output of the first counter can be used as the start condition of the second counter by connecting it directly to the ENP and ENT terminals of the second counter, and the second counter will not start working until the RCO of the first counter outputs a high level. As for the third counter, it is used to count the 9th and 10th bits, and its start condition is that the counter counts to 1111111111, so it needs the RCO of the first two counters to be high at the same time for the third counter to start. Obviously, at the same time for a high level in order to output a high level of this condition in line with the logic of the gate, so the first two counters of the RCO as a while into the input and the gate, the output of the gate connected to the third counter of the ENP and ENT terminals, you can achieve the 9th and 10th bit counting.

In addition, since the 10-bit binary counter can count up to 1023, a clearing device is needed to control the counted range from 0 to 799. The binary form of the 799 is 1100011111, and since this counter needs to be synchronously cleared, our clearing logic is to clear the counter when the first time bits 1-5 and 9-10 of the output binary number are both high. Implemented into specific circuits, the same can be done with an and gate. I will counter the output of the binary number of bits 1-5 and 9-10 as the 5-input and gate and 2-input and gate input, and the output of both as an input to an and non-gate, and the output of the and non-gate are connected to the three counters of the clearing terminal, and thus the logic can be achieved: when the first

time these 7 bits are high at the same time, the clearing terminals of the three counters are low, that is, the three counters are simultaneously clear. Above all, this circuit implements the counting of 0-799.



P11 The circuit of Hcounter

Having completed the design of the 10-bit binary counter, I will consider how to get the corresponding HBlank and HSync waveform. For the waveform of this square wave of HSync, we can understand its logic of one cycle as follows: when the counter counts to the 96th number, i.e., counts to the binary form of the decimal number 95, the output is high and remains in the hold state; when the counter counts to the 800th number, i.e., counts to the binary form of the decimal number 800, the output changes to a low level and remains in the hold state. Similarly, for the HBlank waveform, it is sufficient to replace 96 and 800 in the above logic with 144 and 784, respectively.

Based on the above theory, the JK trigger(rising edge trigger) is a good choice. By consulting the reference manual, the basic logic of the JK trigger can be obtained as shown below.

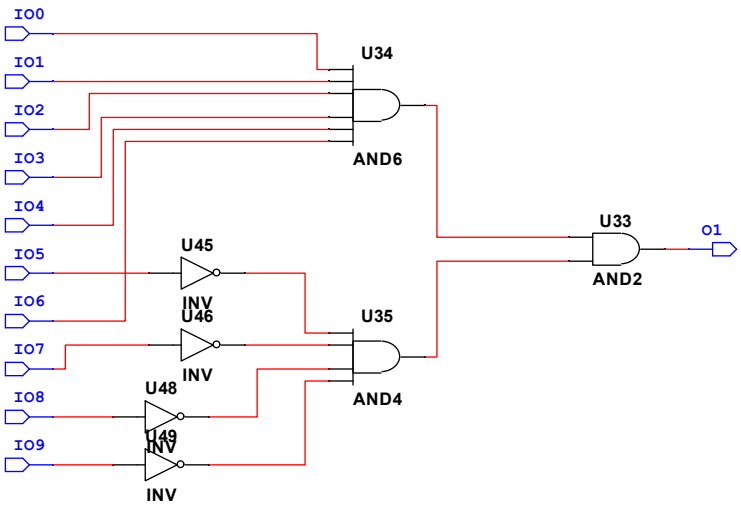
INPUTS					OUTPUTS	
$\overline{PR}$	$\overline{CLR}$	CLK	J	K	Q	$\overline{Q}$
L	H	X	X	X	H	L
H	L	X	X	X	L	H
L	L	X	X	X	H^+	H^+
H	H	↑	L	L	Q_0	$\overline{Q_0}$
H	H	↑	H	L	H	L
H	H	↑	L	H	L	H
H	H	↑	H	H	TOGGLE	

P12 Basic logical of JK flip-flop

Therefore, the JK flip-flop starts operating when both the preset and clear terminals are high. Meanwhile, provided that the input clock is on a rising edge, when J terminal is high level and K terminal is low level, Q output terminal outputs high level; when J terminal is low level and K terminal is low level, Q output terminal is low level.

According to the above logic, the basic idea of my circuit is as follows. As to the HSync signal, provided that the counter and the JK flip-flop have the same clock input, when the counter counts to 95, a pulse signal is given to the J input, at this time the J input is high, the K input is low, and the Q output is high; when the counter counts to 799, a pulse signal is given to the K input, at this time the J input is low, the K input is high, and the Q output is low; when the counting is between 95-799, J and K are both low, and the Q output is in the hold state, so it stays high; similarly, when the counting is between 0-95, J and K are both low, and the Q output segment is in the hold state, and stays low. As to the HBlank waveform, it is sufficient to replace 96 and 800 in the above logic with 144 and 784, respectively.

The key to the above theory is how to get the corresponding pulse signals. My sub-system 96,800,144,784 is the circuit that implements the pulse signal. Since the basic logic of the four circuits is identical, this essay uses 96 as an example. My pulse signal circuit is shown below, which has 10 input connected to 10-bit Binary Counter, 1output connected to J input as well as a series of logic gates. The basic function of the circuit is to output a high level when the counter reaches 95 and a low level for the rest. Specifically, The binary form of 95 is 1011111, therefore the bits 1-5 and 7 of the input are used as inputs to a six-input versus gate, and the rest of the inputs are used as inputs to a four-input versus gate by passing them through a non-gate respectively, and subsequently the outputs of both are used as inputs to a two-input versus gate, and subsequently the logic is obtained that the outputs output a high level when the first occurrence of 0001011111 occurs on the inputs, and a low level for the rest of the cases.

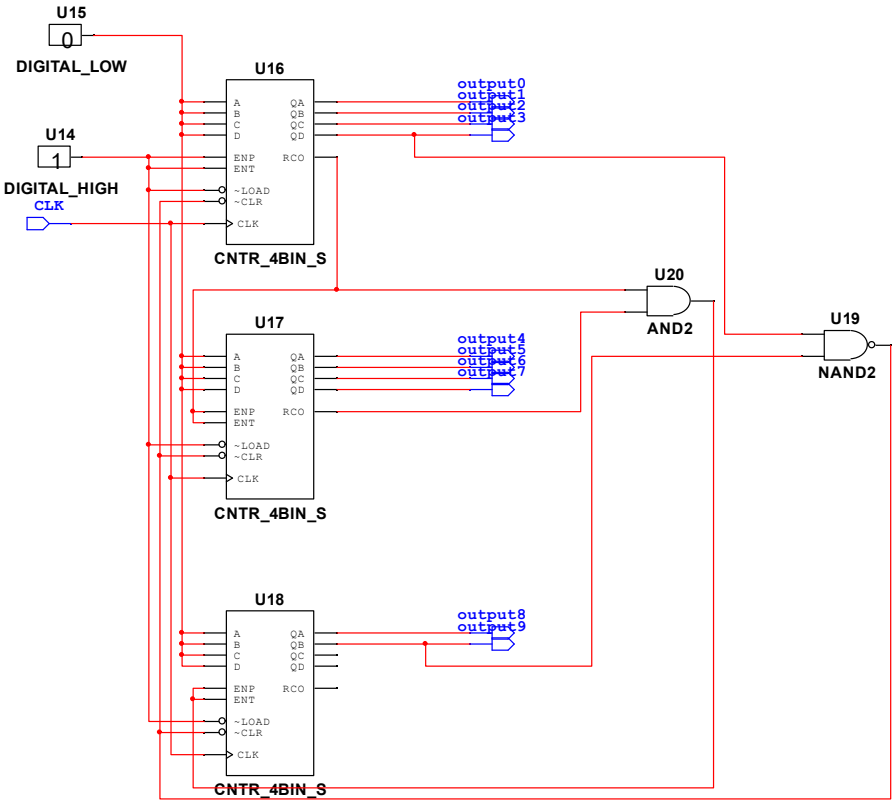


P13 The circuit of '96'

The remaining three circuits are built in the same way, while the inputs are all connected to 10-bit binary counters.The specific wiring is shown in Picture 11.

To sum up, the circuit of horizontal timing is completed, which has one clock input and two outputs as HSync signal and HBlank signal respectively. According to the theory of VGA timing, the clock signal at the input is 25MHz, so in the top circuit, this clock input is connected to CLOCK_A, the 25MHz output of the Clock Divider. Seeing Picture 7 for details.

Similarly, we use the same logic to build the vertical timing circuit. The sub-system VSYNC is a ten-digit synchronized binary counter capable of counting a total of 521 numbers (Although the 10th position is not useful here); sub-systems 31,511,2,512 are the start position or end position of the high level of the VSync signal and the VBlank signal, respectively. Below is the logic circuit diagram of VSYNC, the rest of the circuits are not shown here as the logic is more or less the same.



P14 The circuit of Vcounter

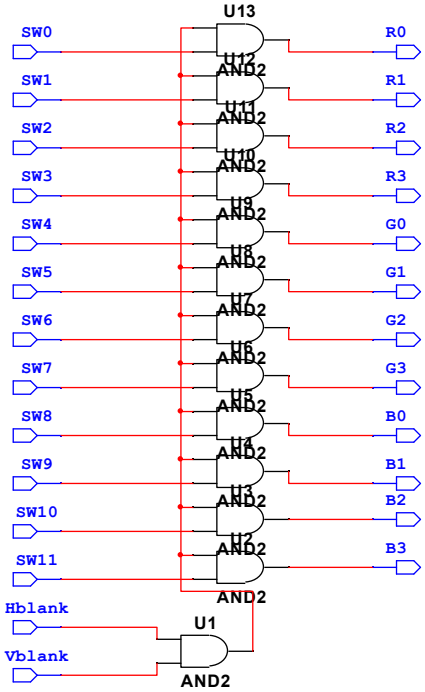
The point of difference is that the input clock of Horizontal timing is a pixel clock with a period of 0.04us, whereas the input clock of Vertical timing is HSync with a period of 32us, so after completing the construction of the circuit for Horizontal and Vertical, the input HSync of Horizontal timing is used as the Vertical timing input clock.

2.2 RGB circuit

Compared with driving circuit, RGB circuit is relatively simple. The whole circuit consists of 14 inputs, 12 outputs and 13 2-input AND gates. The overall logic of the circuit is to use 12 switches to control the 12 bits of the RGB colour so that the screen renders the appropriate colour according to the switches, but only renders the colour (the whole screen) in the raster area selected by the blank signals.

The Basys3 board uses 14 FPGA signals to create a VGA port with 4 bits per colour and two standard sync signals (HSync and VSync) which can subsequently be generated according to its internal circuitry. Therefore, the logic circuits can be built according to the digital to analogue signals.

The logic of the 2-input AND gate is that the output is high only when both inputs are high, otherwise they are both low; and in the case where both inputs are signals, the output is the portion of the two signals that are both high.Therefore, I combine the HBlank and VBlank signals into a blank signal through a AND gate, and then I perform a sum operation on the resulting blank signal with each of the switch inputs, and use the result of the operation as the real RGB signal, so that I can output the corresponding RGB signal to the grating in the range selected by the blank signal.

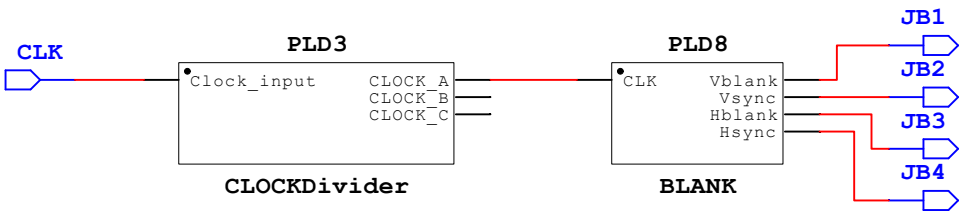


P14 The circuit of RGB

3 Testing

3.1 SYNC/BLANKING testing

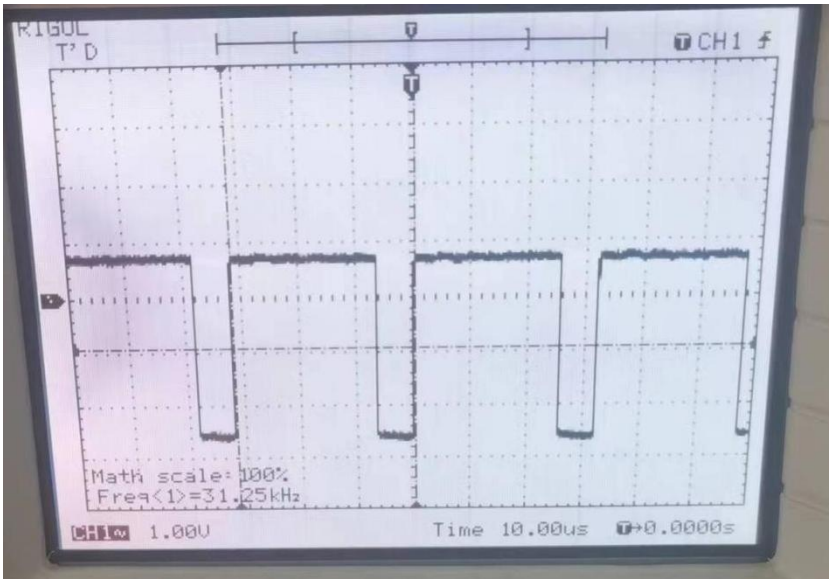
For SYNC and BLANKING output signal testing, a total of 4 quantities need to be tested, i.e. HSync, VSync, HBLank, VBlank. These four waveforms can be viewed more clearly using an oscilloscope and judged to be correct. Again, similar to Stage1, I built the appropriate test circuit and used JB1-JB4 as the outputs for each of the four signals. Below is the test circuit diagram.



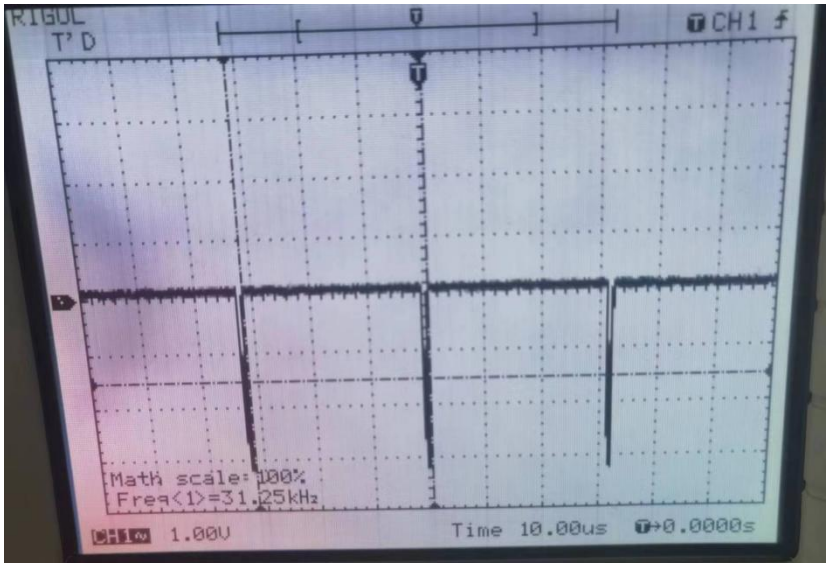
P15 The testing circuit of output signals

Subsequently, after generating a VHDL file of the circuit and burning it to the Basys3 board, the four output waveforms can be obtained by connecting an oscilloscope to the corresponding pins and GND, and the results are as follows.

As to horizontal timing:



P15 HBlank signal



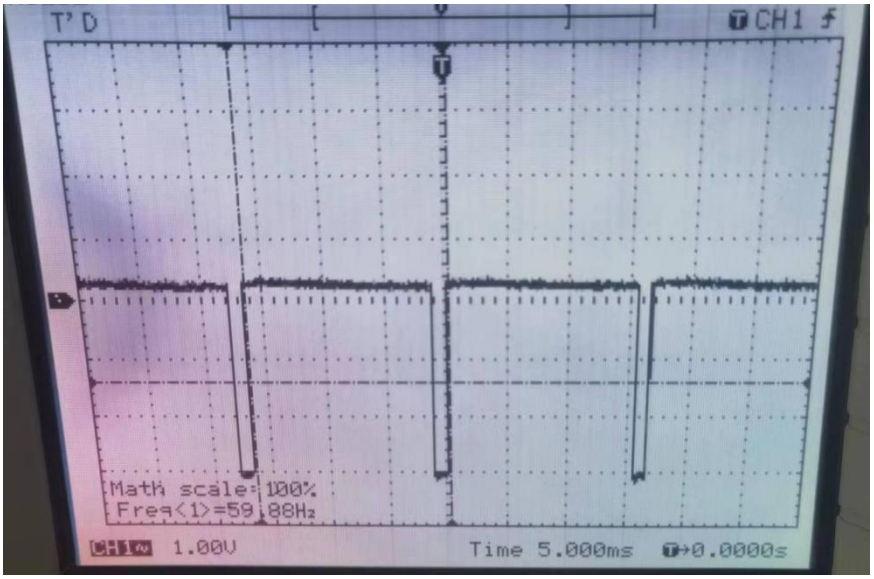
P16 HSync signal

The theoretical frequency of the output waveform can be simply calculated here.

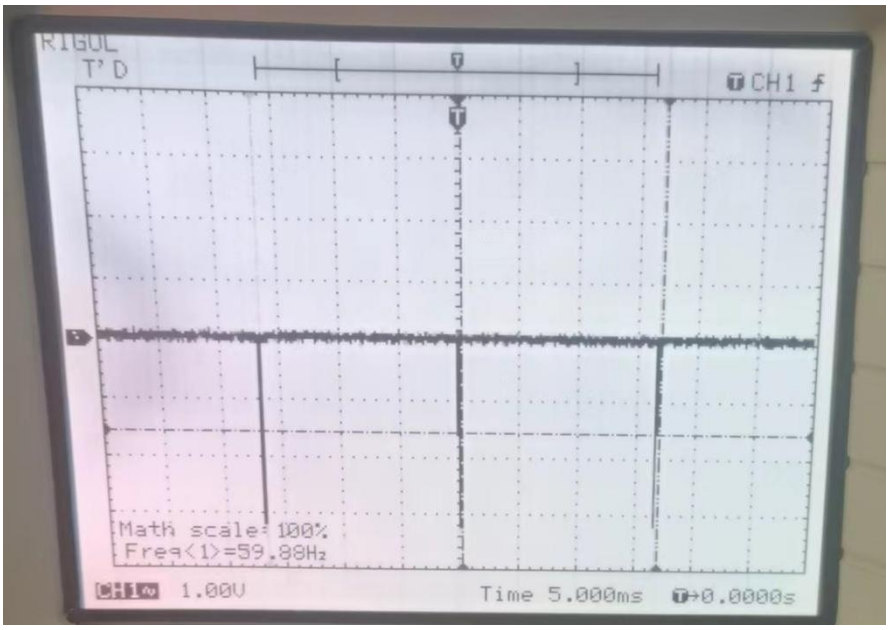
$$f_{th} = \frac{1}{800 \times 0.04\mu s} = 31.25\text{kHz}$$

According to the above oscilloscope results, we can see that the overall waveform high and low level transformations are more obvious, is a more standard square wave, and at the same time, the frequency is almost the same as the theoretical frequency of 31.25kHz; at the same time, the duration of the high level of the HSync is significantly longer than that of the HBLANK, and therefore, the overall results of the experiment are better.

As to vertical timing:



P17 VBlank signal



P18 VSync signal

The theoretical frequency of the output waveform can be simply calculated here.

$$f_{tv} = \frac{1}{521 \times 32\mu s} = 59.98\text{Hz}$$

According to the above oscilloscope results, we can see that the overall waveform high and low level transformations are more obvious, is a more standard square wave, and at the same time, the frequency is almost the same as the theoretical frequency of 59.98Hz; at the same time, the duration of the high level of the VSync is significantly longer than that of the VBLANK, and therefore, the overall results of the experiment are better.

Above all, all four waveforms have a high degree of similarity to the theoretical waveforms, while the frequencies are normal, so the experiment is overall more successful.

3.2 RGB and VCOUNT/HCOUNT testing

For the testing of RGB and of VCOUNT/HCOUNT, we can judge the correctness of the Basys3 board directly from its experimental results. The testing video is Stage2_RGB_and_VCOUNT_HCOUNT_testing.

Through the video we found that when all switches are low, the display shows a blank screen; when SW0-SW3 become high and the rest are low, the whole screen becomes red; when SW4-SW7 become high and the rest are low, the whole screen becomes green; and when SW8-SW11 become high and the rest are low, the whole screen becomes blue.

As a result, VCOUNT and HCOUNT are able to count correctly, while the RGB circuit is designed successfully.

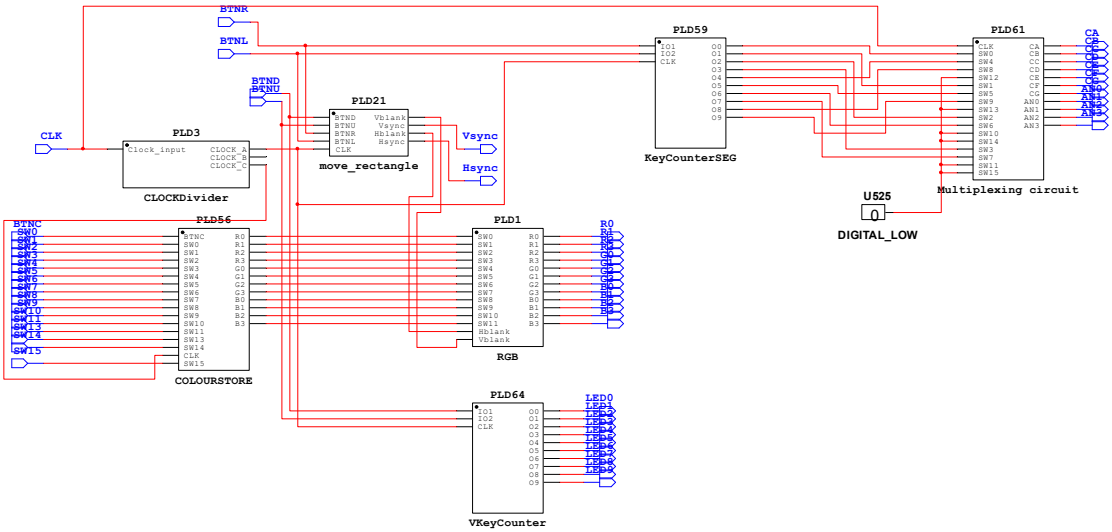
Stage3 Dynamic Programmable Video Test Card

1. Introduction

In this stage, the main task is to realize the Dynamic Programmable Video Test Card, which consists of four parts: colored rectangle, moving rectangle, coordinate recording, as well as 2colors storage and 2 coloring modes. These four components fulfil the following functions: The rectangle presented in the screen appears at the top left corner and can be moved up, down, left and right using BTNU, BTND, BTNL, BTNR, and cannot be moved outside the boundary. At the same time, while moving, the 7-segment display and the LEDs on the Basys3 board present the horizontal and vertical coordinates of the point in the top-left corner of the rectangle, respectively; For the colour storage part, the colour of the rectangle can be determined by SW0-SW11 and the two selected colours can be stored by BTNC according to SW13 and SW14, at the same time, two different colour selection modes can be selected according to the high and low levels of the output level of SW15, mode1 is that SW15 switches a colour by pressing the BTNC once when it is at a low level, mode2 is that SW15 switches a colour by pressing the BTNC when it is at a high level and mode2 is that SW15 switches a colour by pressing the BTNC once when it is at a high level. When SW15 is at high level, the displayed colour will be switched according to the two stored colours alternately at a frequency of 0.5Hz periodically.

2. Circuit Design

The following schematic shows my Dynamic Programmable Video Test Card. I use a hierarchical structure to divide the circuit into seven main sub-systems.



P19 Dynamic Programmable Video Test Card

Among them, Clock Divider is the design of Stage1, Multiplexing circuit is the design of 7-segment display in Stage1, and RGB is the design of Stage2, which will not be repeated here.VKeyCounter and KeyCounterSEG record the vertical and horizontal coordinates of the upper-left corner of the rectangle, respectively, while there is a boundary effect, the specific boundary range will be described in detail

subsequently. The movement of the rectangle is mainly implemented in move_rectangle, which has this logic structure and inputs and outputs similar to the timing circuit in Stage2. As to the COLORSTORE, it enables storing two rectangle colours and switching between mode1 and mode2. The sub-system are described in detail below.

2.1 Colored rectangle Design

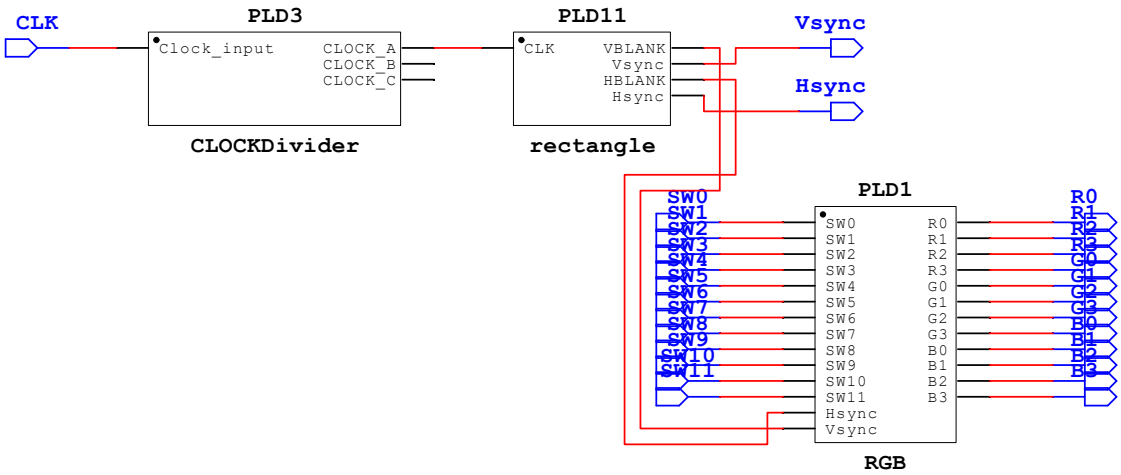
In this section I will only draw the required rectangle in the top left corner of the screen, the rest of the functionality will improve the circuit on subsequent requests.

Firstly, calculating the height and width of the rectangle according to my Essex ID and the requirements of the experiment. My Essex ID is 2107294, so the height and width can be calculated by the following functions:

$$W = 4 \times 73 + 2 \times 94 = 480$$

$$H = 4 \times 37 + 2 \times 49 = 295$$

According to the circuit of Stage2, we only need to change the HBlank and VBlank in the timing circuit to get the corresponding rectangle. Therefore, the total circuit for drawing the rectangle is as follows, which is similar to the circuit of Stage2.

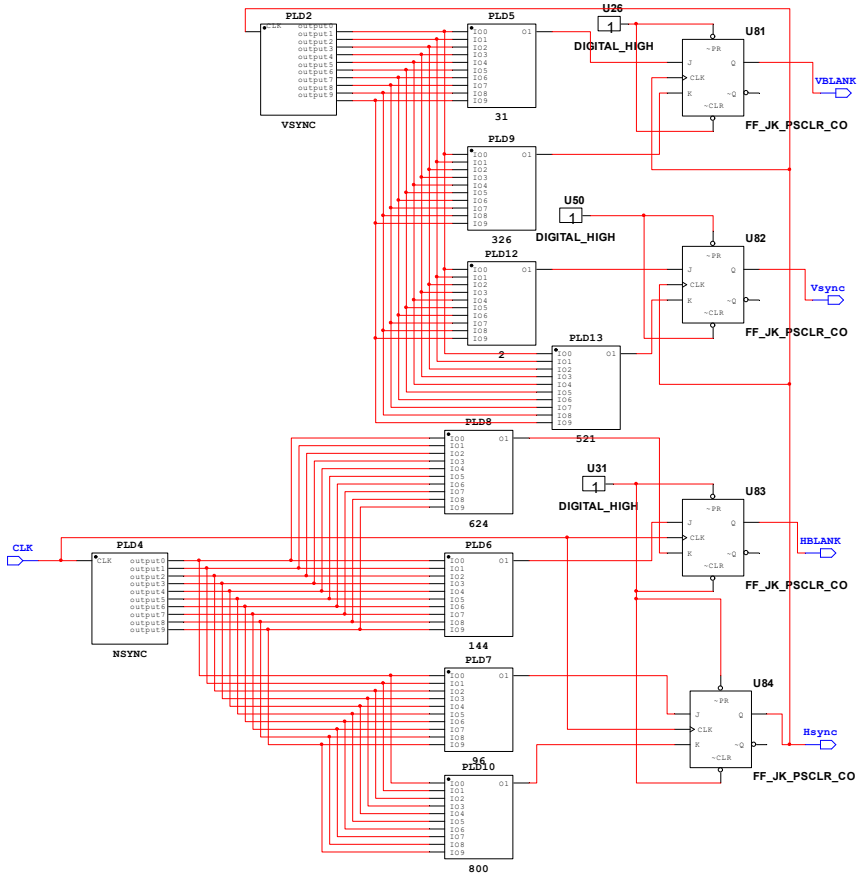


P20 Circuit of initial rectangle

The difference is that I replaced the BLANK sub-system with a rectangle sub-system, and here is the exact circuit for the rectangle sub-system. The circuit is also similar to the timing circuit and shares the same principles. For Stage2, we can interpret the experimental result it as generating a 640×480 coloured rectangle, so in order to get a 480×295 coloured rectangle, all that is needed is to change the high level length of the HBlank signal from 640 to 480, and the high level signal of the VBlank signal from 480 to 295. at the same time, since the rectangle exists in the upper left corner of the At the same time, since the rectangle exists in the upper left corner, when changing the length of the high level of both signals, you only need to change the end position of the high level and not the beginning position of the high level.

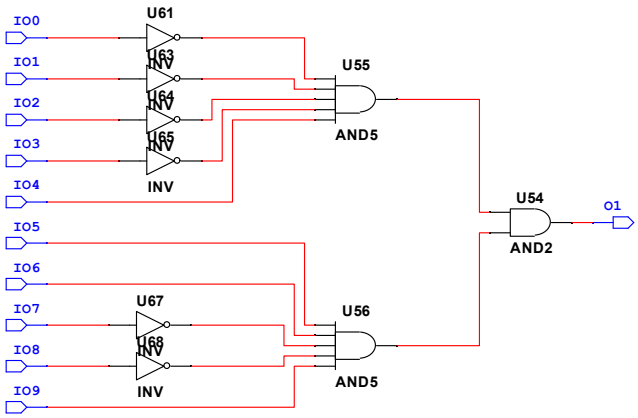
Specifically, According to the logic of this circuit, it is necessary to change the pulse signals at the J input and K input of the JK trigger. Since no changes are required for the HSync and VSync signals, the 96,800,2,521 subsystems can be

retained for continued use. For the HBlank and VBlank signals, the starting position of the high level does not need to be changed, so the sub-systems 144 and 31 can also be retained; then, depending on the width of the rectangle required, the sub-system 784 is changed to the sub-system 624, i.e., a pulse signal is given when the counter reaches the digit 623, and similarly, depending on the height of the rectangle, sub-system 511 is changed to 326 sub-system, i.e., to give a pulse signal when the counter counts to the number 325.



P20 Circuit of rectangle

The following section shows the circuit diagram of the 624 sub-system, and the rest of the circuit diagrams will not be shown since they are the same as the Stage2 principle.



P21 Circuit of 624

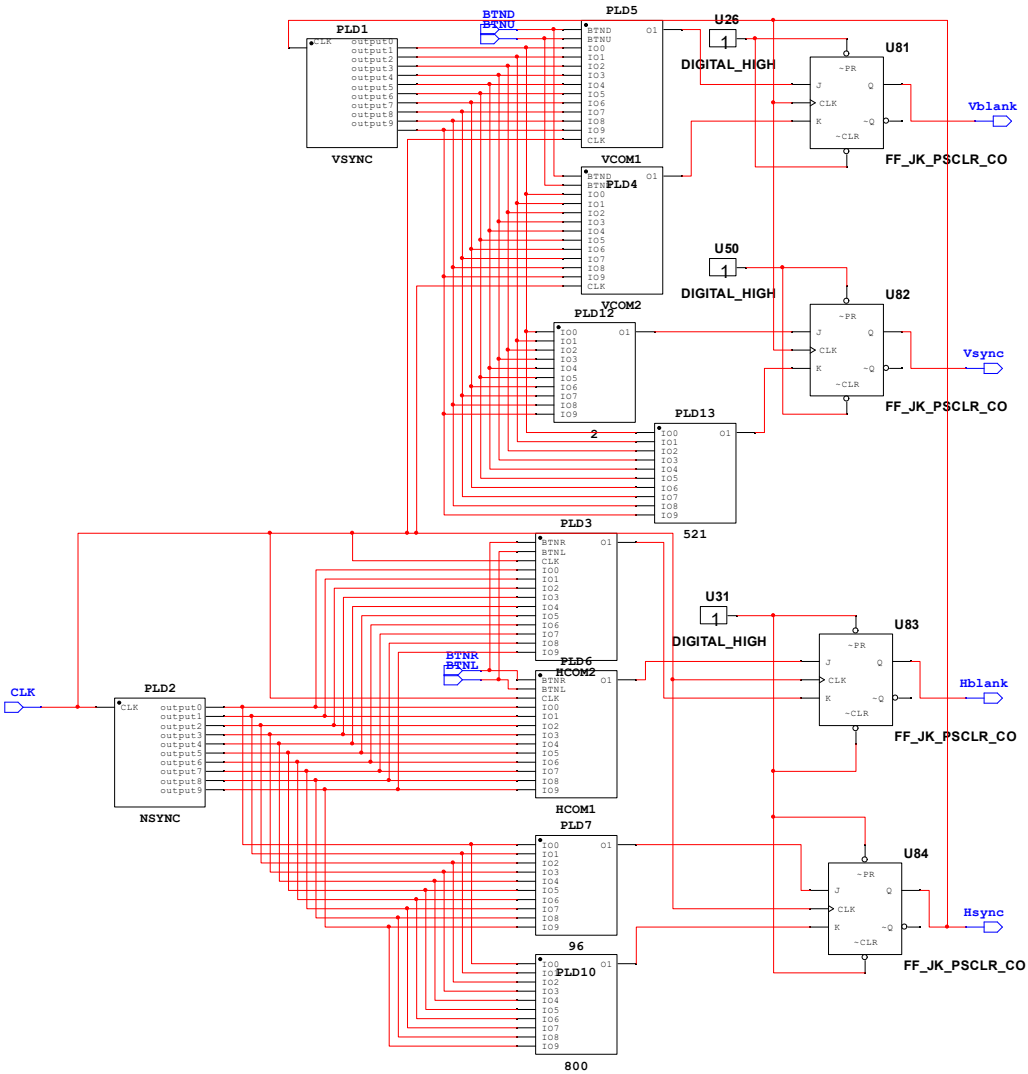
2.2 Moving rectangle Design

2.2.1 General introduction

The circuit design in section 2.1 only allows for a static rectangle to be drawn, not moved, so in this section, changes will be made to the above circuit.

For moving rectangles, it is also necessary to make changes to the HBlank and VBlank signals and retain the original HSync and VSync signals to ensure that the screen renders the rectangle properly. As to the change of HBlank and VBlank, the general idea of this paper is to design a counter driven by the button, according to the number of times the button is pressed to determine the start position of the high level of the Blank signal, and at the same time, since the width and height are fixed values, the end position can also be determined.

The following schematic shows my improved Circuit, which replaced the original 144,624,31,326 sub-systems with HCM1, HCM2, VCOM1, VCOM2.



P22 Circuit of move_rectangle

These four circuits share the same principle, and here we use HCOM1 as an example. The internal part of HCOM1 consists of a 10-bit binary counter, an adder and a comparator.

The counter counts the number of times the button is pressed, i.e. when BTNR is

pressed, the number counted by the counter is increased by one, and when BNTL is pressed, the number counted by the counter is decreased by one. At the same time, due to the existence of the boundary effect, when the rectangle moves to the leftmost part of the screen when the BTNL is pressed, the number counted will not be reduced, and similarly when the rectangle moves to the rightmost part of the screen when the BTNR is pressed, the number counted will not be increased.

For the adder, since pixel number 143 corresponds to the starting position of the HBlank signal high when the rectangle is in the upper left corner, the pixel number corresponding to a key press can be calculated using the following formula (both horizontal and vertical are given):

$$\begin{cases} N_{HADD1} = 143 + k_1 \\ N_{HADD2} = 623 + k_1 \\ N_{VADD1} = 30 + k_2 \\ N_{VADD2} = 326 + k_2 \end{cases}$$

Where k_1 is the number output by the counter in the horizontal direction after BTNR and BTNL have been pressed a number of times; k_2 is the number output by the counter in the vertical direction after BTNU and BTND have been pressed a number of times. Therefore, the adder completes the corresponding addition calculation here to facilitate the subsequent circuit to output the correct pulse signal.

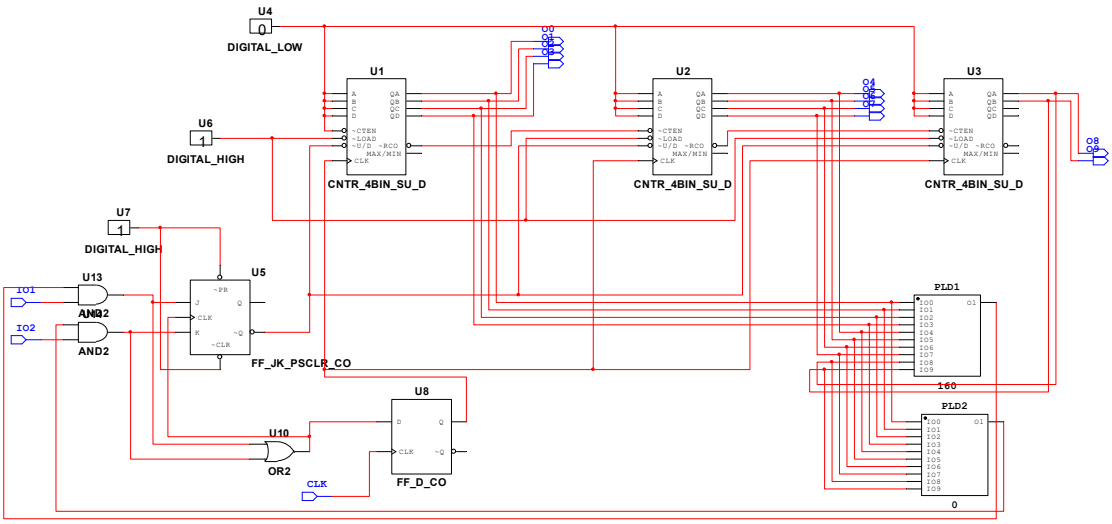
When it comes to the comparator, It is used to compare the output of the adder with the number output by the counter HSYNC after connecting the corresponding pixel clock, and when the two numbers are exactly the same, a pulse signal is output.

Similarly, for HCOM2, VCOM1, VCOM2 have the same principle, the only difference is the different numbers added by the adder and the different range of the boundaries. After completing the above design, the rectangle can be moved to the appropriate position according to the circuit of the initial rectangle.

2.2.2 Functional and logical description of individual circuit

Here we use HCOM1 as an example.

① **horizontal button counter**



P23 Circuit of horizontal button counter

The above schematic shows my horizontal button circuit, which consists of 3 Synchronous UP/DOWN Modulo 16 Binary Counter, one JK flip-flop, one T flip-flop, two AND gates and two sub-systems called 0 and 160 with one clock input two button inputs and 10 binary number outputs.

3 Synchronous UP/DOWN Modulo 16 Binary Counter are the key part of the whole counter circuit, which implements 10-bit binary counting. As to Synchronous UP/DOWN Modulo 16 Binary Counter, it will perform the counting function when the LOAD terminal is high and the CTEN terminal is low, and counts up on the rising edge of the clock when the U/D terminal is low, and counts down vice versa. Also, it has the same basic counting function as the counter in Stage2. Similarly, according to the design principle of the 10-bit counter in Stage2, the next counter is activated when the output of the first counter is 1111, so the above function can be accomplished by connecting the RCO terminal of the former to the CTEN terminal of the latter in the process of cascading. Since the counter is driven by a BUTTON, either BTNR or BTNL, and either counter starts to work when pressed, the two BUTTON inputs are used as the inputs to a two-input or gate, and the output of that gate is used as the clock for the three counters and the JK flip-flop.

The JK flip-flop here is to decide whether the counter is for upward or downward counting. The basic principle is to connect the outputs to the U/D terminals of the three counters, and output a low level when IO1(Connecting BTNR at the upper circuit level) is pressed and a high level when IO2(Connecting BTNL at the upper circuit level) is pressed; meanwhile, due to the existence of the boundary, the button can only affect the outputs of the JK flip-flop if the rectangles are not moved to the boundary. For boundary effects, the range of movement is first calculated here:

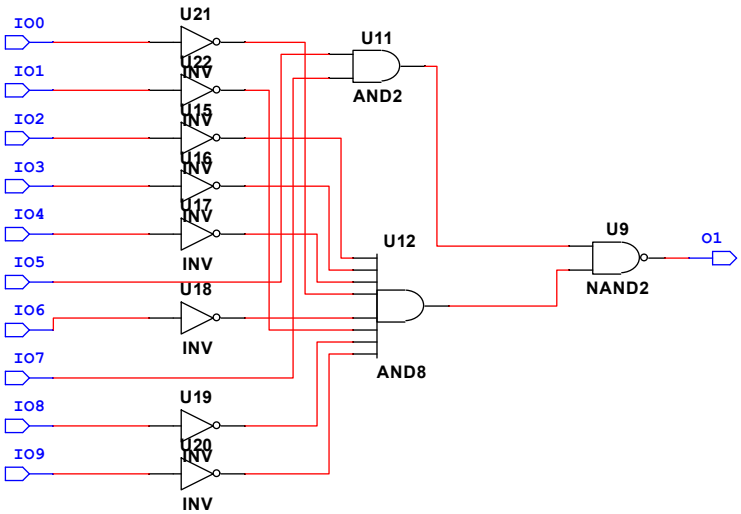
Horizontal range:

$$784 - 624 = 160 \ \rightarrow \text{range}[0,160]$$

Vertical range:

$$511 - 326 = 185 \ \rightarrow \text{range}[0,185]$$

Therefore, Here the 160 and 0 sub-systems represent the upper and lower boundaries, respectively, when moving. The specific circuit is shown below (160 as an example).

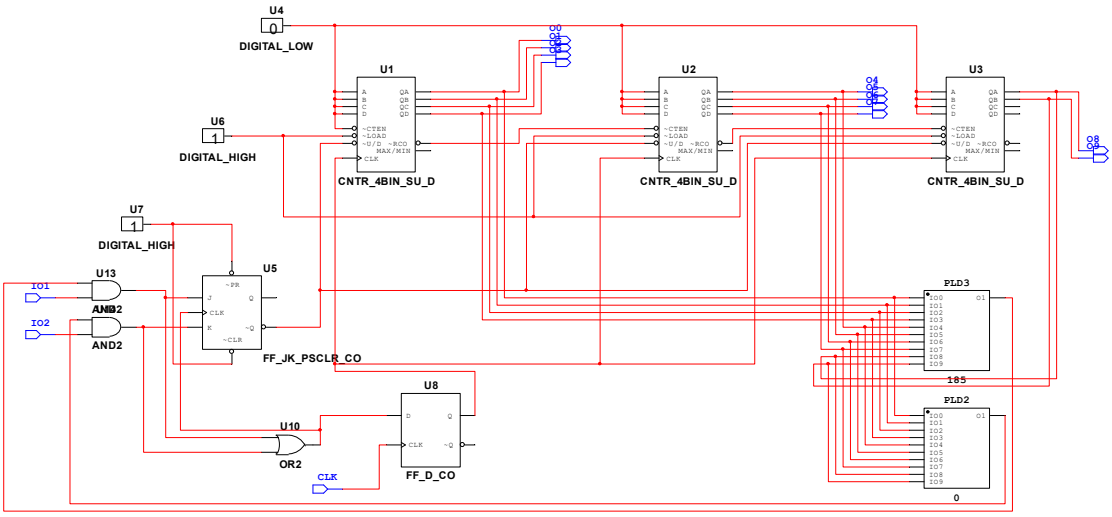


P23 Circuit of 160

The 10-bit binary form of 160 is 0010100000, so the 10-bit binary output of the counter is used as an input to the above, and the output of this circuit results in a high level when the output of the count is not in the binary form of 160, and the circuit outputs a low level only when the counter outputs 160. Similarly, the principle of sub-system 0 is the same. Meanwhile, according to the operation logic of JK flip-flop, when J inputs high level and K inputs low level, the output of non-Q terminal is low; when J inputs low level and K inputs high level, the output of non-Q terminal is high, which is exactly in line with the logic of key input and U/D input. Subsequently, by using the output of the 160 subsystem with IO1 as an input to a two-input-and-gate, connecting the output of this gate to the J input of the JK flip-flop, and using the output of the 0 subsystem with IO2 as an input to a two-input-and-gate, and connecting the output of this gate to the K input of the JK flip-flop, upward counting is carried out when BTNR is pressed and stops when 160 is counted, and downward counting is carried out when BTNL is pressed and stops when 0 is counted.

However, by saving the circuit as a VHDL file and burning it to the Basys3 board, I found that since the input clock of the JK flip-flop is the same as the input clocks of the three counters, but the inputs at the U/D side, due to the logic operation of one of the JK flip-flops, produce a certain delay and start up slower than the input clocks of the counters, thus a counting error occurs at the time of the change from up count to down count! Therefore, a counting error occurs when counting up changes to counting down. Therefore, in order to equalize the delay between the U/D side of the counter and the clock input, a D flip-flop is passed through before connecting to the clock input of the counter via an or-gate, which then results in the same delay for both, and the counting is actually realized. For the D-flip-flop the clock input can be arbitrary, as long as the frequency is greater than the frequency of the key input, which is well realized since the keys are manually pressed by a human. Here we use 25MHz of Clock Divider as its input.

As to vertical button counter, the principle is the same as Horizontal Button Counter, just change the boundary, only the circuit is shown here. No more repetitive explanations



P24 Circuit of vertical button counter

②adder

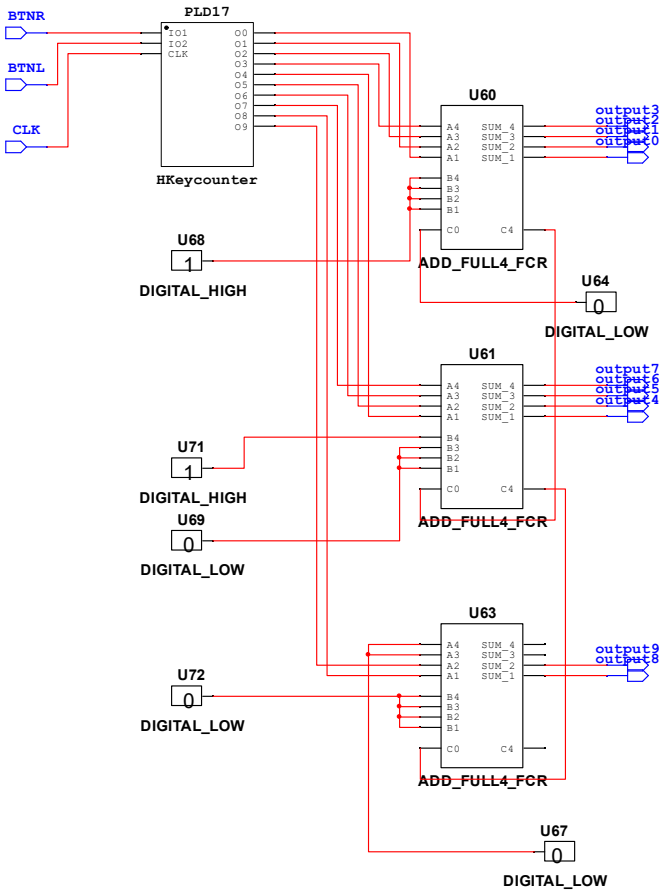
The following schematic shows my adder, which is a 10-bit binary adder consisting of three 4-bit full adder and the horizontal button counter described above. As to the 4-bit full adder, its basic principle of operation is to add two four-bit binary numbers (A1 and B1 are LSB), to get the corresponding four-bit binary output, if there is the result of the sum is greater than 1111, then C4 output a high level to indicate a rounding; C0 is for the rounding of the case of the preparation of the input for the connection of the last level of the rounding of the C4. Since we are using HCOM1 as an example, the addition operation to be implemented here is:

$$N_{HADD1} = 143 + k_1$$

Therefore, Connect the output of the counter to part A of each full adder, connect the 10-bit binary form of 143, 0010001111, to part B of each full adder, and then cascade the three full adders in accordance with C4 connected to C0 respectively (C0 of the first full adder is connected to a low level), and then finally complete the 10-bit adder by selecting 10 bits of the 12 outputs.

Above all, the inputs of the whole circuit are the inputs of the counter and the outputs are the outputs obtained after the addition operation, which are used in the subsequent circuits to generate the pulse signals.

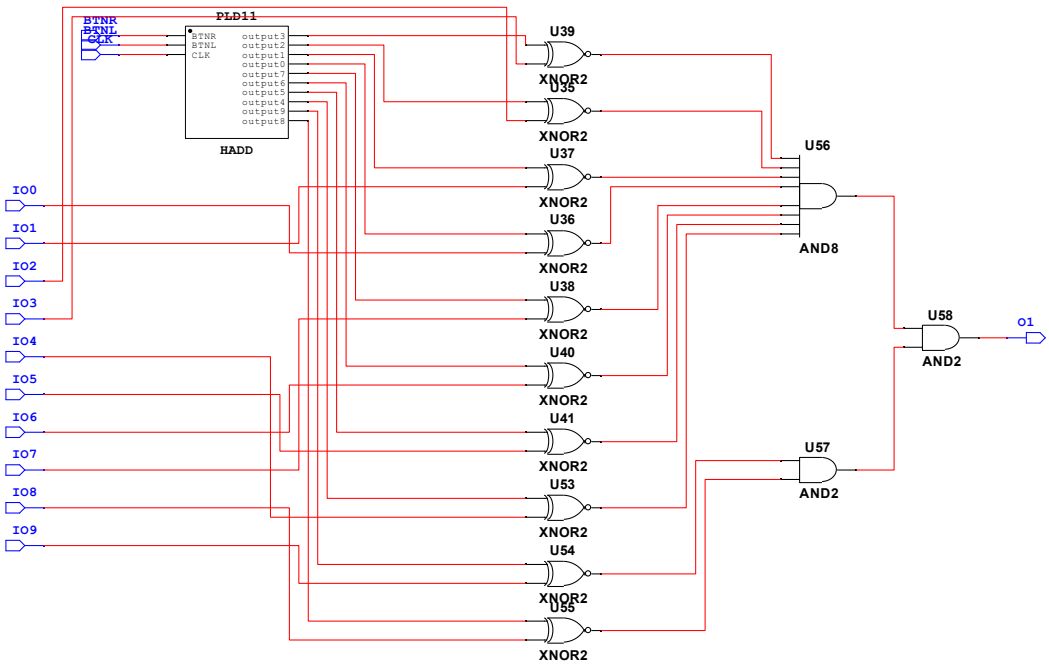
The principle of the remaining three adders is the same, but the numbers added become 623,30,326.



P25 Circuit of adder

③comparator

The following schematic shows my horizontal comparator, which consists of a adder sub-system and a series of logical gates with 10 inputs and one pulse input. These 10 inputs are used to connect the level counter in the timing circuit. When the counter counts from 0 to 799, a high level is output from the output if a number identical to the binary number output by the adder appears. This circuit uses 10 two-input XNOR gates to implement the comparison of numbers and the judgement that the numbers are the same: For the two-input XNOR gate, when both inputs are high or both low, the output is high, and vice versa two inputs are high and low, the output is low; therefore, according to the characteristics of the gate, two ten-bit binary numbers of the same bit of the digits are connected to the same XNOR gate for bit-by-bit comparisons, followed by the output of the 10 XNOR and then do an AND operation to get the corresponding output results. Thus it is guaranteed that this circuit will only output a high level pulse signal when all 10 XBOR outputs are high.



P26 Circuit of comparator

Thus based on the same principle we get four comparator circuits: HCOM1, HCOM2, VCOM1, VCOM2.Subsequently, the moving rectangle can be realized by replacing them as subsystems with subsystems 144,624,,31, and 326 of the circuits of Section 2.1, respectively.

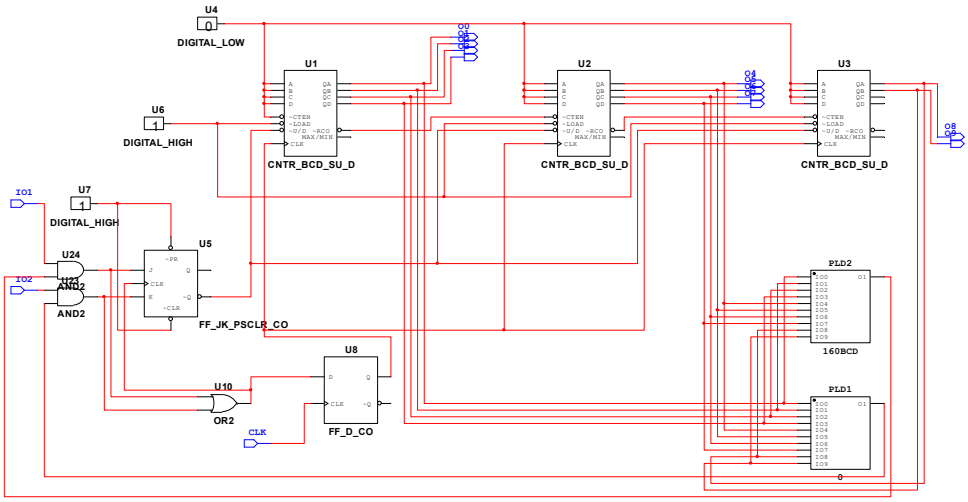
2.3 Coordinate recording Design

Displaying the coordinates of the top-left corner of the rectangle is essentially a visual output of the counters from the previous section. Therefore, the Horizontal Button Counter and Vertical Button Counter can continue to be used. Also, the multiplexing circuit in Stage1 can continue to be used when recording transverse coordinates, simply by replacing the original switch input with the current coordinate input. Based on the calculations in the previous section, the range of horizontal coordinates is [0,160] and the range of vertical coordinates is [0,185].

2.3.1 Horizontal coordinate recording

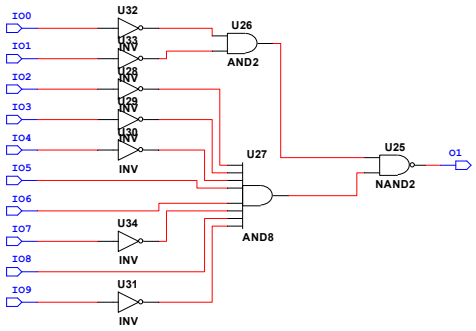
Since the horizontal coordinates need to be displayed with a 7-segment display, it is not possible to connect the 10 outputs of the Horizontal button counter directly to the multiplexing circuit in Stage1, because the Synchronous UP/DOWN Modulo 16 Since the Synchronous UP/DOWN Modulo 16 Binary Counter can count from 0000 to 1111, i.e. decimal 0-15, and for the 7-segment display, the binary inputs for each four digits can only count from 0000 to 1001, it is necessary to modify the original Horizontal button counter.

The following schematic shows my modified circuit called KeyCounterSEG, which only replaced all CNTR_4BIN_SU_D with CNTR_BCD_SU_D and changes to the 160 sub-system affecting the border to 160BCD sub-system. The essential difference between the two counters is that the 4BIN counts from 0000 to 1111, while the BCD counts from 0000 to 1001, and the latter meets the requirements of the 7-segment.



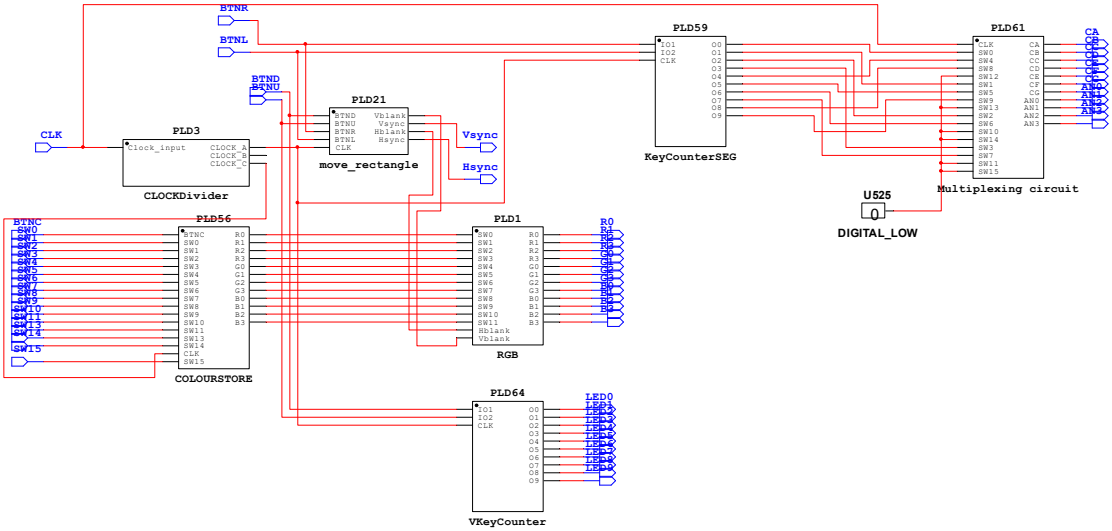
P27 Circuit of KeyCounterSEG

As the type of counter is changed, the arithmetic logic for each bit of output also changes. Fortunately, we only need to count from 0 to 160, which is possible for these 3 BCD counters, and also the highest bit has only two possibilities, 0 and 1, so only two bits need to be used in the last counter, so the overall circuit construction is not changed.For the boundary change, the 0 subsystem can be retained since the result of 0 in 10-bit binary representation is the same as that in BCD, and for the 160 subsystem, it is represented as 000101100000 in BCD (taking the 10 bits to be 0101100000), so the new 160BCD subsystem is as follows.



P28 Circuit of 160BCD

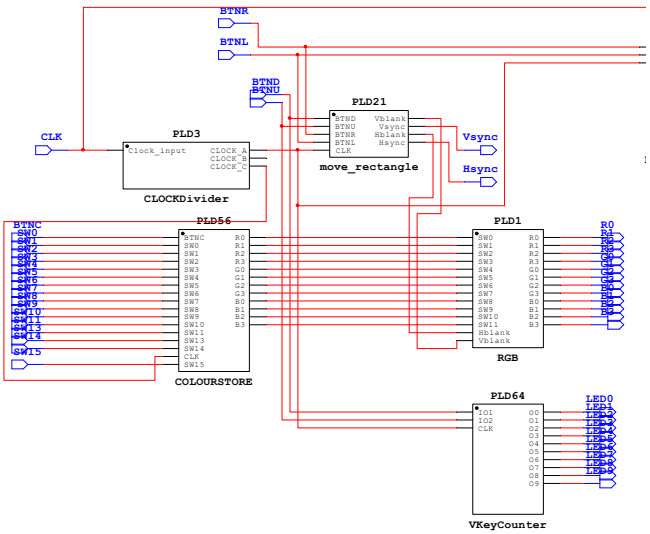
In the top circuit, it is wired as shown below, the counter's 10 for the output corresponds exactly to the multiplexing Circuit's SW0-SW9, and for the redundant SW10-SW15, connecting directly to low level is sufficient.



P29 Dynamic Programmable Video Test Card

2.3.2 Vertical coordinate recording

Since the vertical direction is using LED to display the coordinates, you can directly follow the vertical button counter in the previous section and connect LED0-LED9 directly to the output of this counter to complete the coordinate recording.



P29 Dynamic Programmable Video Test Card

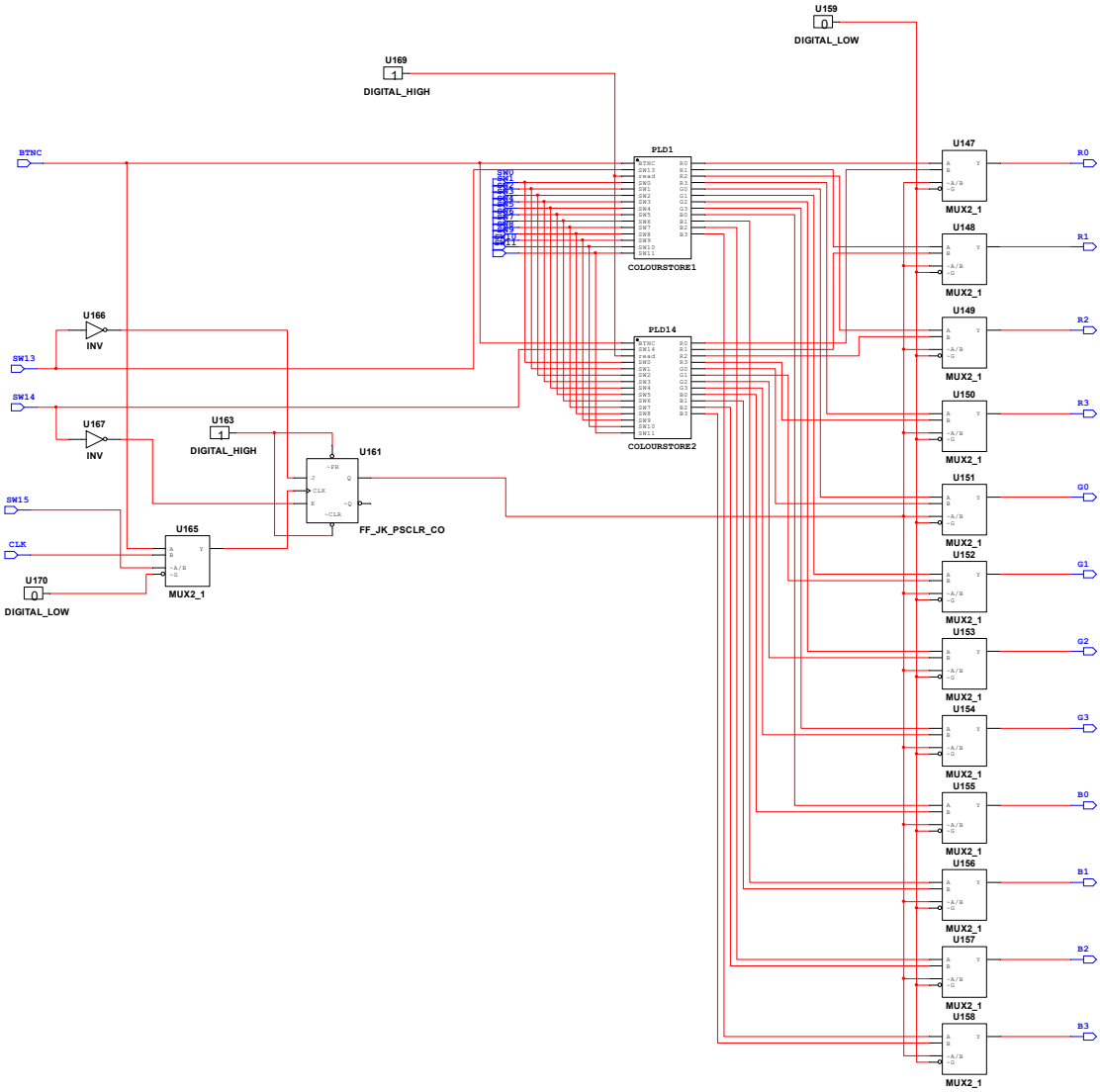
Similarly, in the top circuit, it is wired as shown above.

2.4 2 colors storage and 2 coloring modes Design

For this section, the specific functional logic is as follows: SW13 and SW14 are used to store colour 1 and colour 2 respectively; when SW13 is high and SW14 is low, press the BTNC button, the circuit will store the RGB signals output from SW0-SW11 as colour 1; when SW13 is low and SW14 is high, the circuit will store the RGB signals output from SW0-SW11 as colour 2; when SW13 is low and SW14 is high, the circuit will store the RGB signals output from SW0-SW11 as colour 2.

SW13 and SW14 are both high, this is undesirable and should be avoided in the circuit with logic. At the same time, after storing the two colors in play, SW13 and SW14 are all set to a low level, at this time it will switch the colour mode according to SW15, SW15 out of the low level for mode 1, press once BTNC will switch the colour of the current rectangle once, frequent presses are two colours to switch over and over again; when SW15 comes out high for mode 2, the colour of the rectangle will keep switching colors automatically at a frequency of 0.5Hz, producing a colour flashing effect.

The overall logic circuit is shown below, which consists of 13 MUX2_1 and 2 sub-systems about color storage as well as some logical gates and one JK flip-flop with one button input, 3 switch inputs and one clock input as well as 12 color outputs.



P30 Circuit of color storage

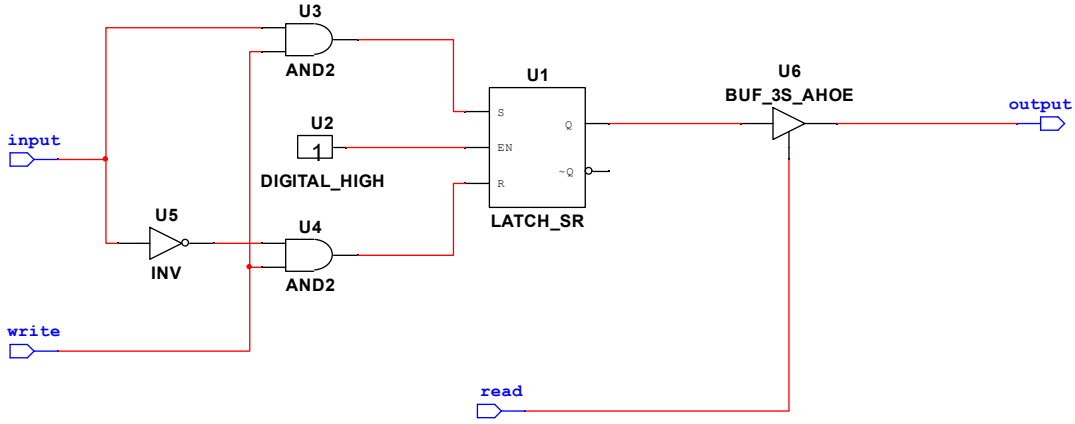
The whole circuit can be divided into 3 important parts: the 1-bit register, the single-colour storage circuit, and the mode selection circuit.

2.4.1 1-bit register

The 1-bit register is used to record 1 bit of the 12-bit input RGB signal and still

give the corresponding signal in memory after the input is changed, this circuit uses 12 1-bit registers to achieve the storage of 12-bit RGB signals.

The following schematic shows my 1-bit register, which has one RS flip-flop and a series of logical gates with one signal input and two functional input as well as one signal output.



P31 Circuit of 1-bit register

For the RS flip-flop, its logic is shown below. The RS flip-flop starts to operate with the enable terminal high. The RS flip-flop starts to operate with the enable terminal high. The output is in hold state when both R input and L input are low, output Q is low when R is high and L is low, and output Q is high when R is low and L is high, while trying to avoid both being high.

INPUTS			OUTPUTS	
S	R	EN	Q	$\overline{Q}$
X	X	L	Q_0	$\overline{Q_0}$
L	L	H	Q_0	$\overline{Q_0}$
L	H	H	L	H
H	L	H	H	L
H	H	H	Restricted combination	

P32 logic of RS flip-flop

For this circuit, a high level is connected to the enable terminal of the RS flip-flop to keep it in operation at all times; the input terminal is the signal input and the write terminal is the storage signal, using both as inputs to a two-input versus gate and using the output of the versus gate as the input to S. Subsequently, the input is passed through a non-gate and then used again with the write terminal as the input signal to a two-input versus gate, and using the output signal as the input to R. By connecting the above circuits, we are able to achieve the following functions: When the write terminal input is low, the entire circuit is in a holding state, the signal input no matter what signal is input, the R terminal and the S terminal are low, the flip-flop is in a holding state, the output signal depends on the last signal; when the write terminal input is high, the level of the R terminal and the S terminal depends on the signal

input, the input input is high, S is high, the R is low, the output is high, and conversely, S is low, R is high, and the output is low, therefore, the Q output is stored in the signal of the input.

For subsequent circuits, connect the Q output to the input A of a 3-state buffer, connect the read input to the input C of that buffer, and use the output of the buffer as the output of the entire circuit. For the 3-state buffer, it has the following logic:

Input A	Input C	Output Y
L	H	L
H	H	H
X	L	Z

P33 logic of 3-state buffer

According to the above table, it can be seen that when input C is high, the buffer takes the input of input A as an output, so a read function is implemented. Therefore, in this circuit, when a high level is input to the READ input, the circuit starts to perform the read function, and the signal stored in the RS flip-flop is output.

As indicated above, the 1-bit register implements the function of storing and reading the 1-bit input signal for subsequent colour storage.

2.4.2 Single-colour storage

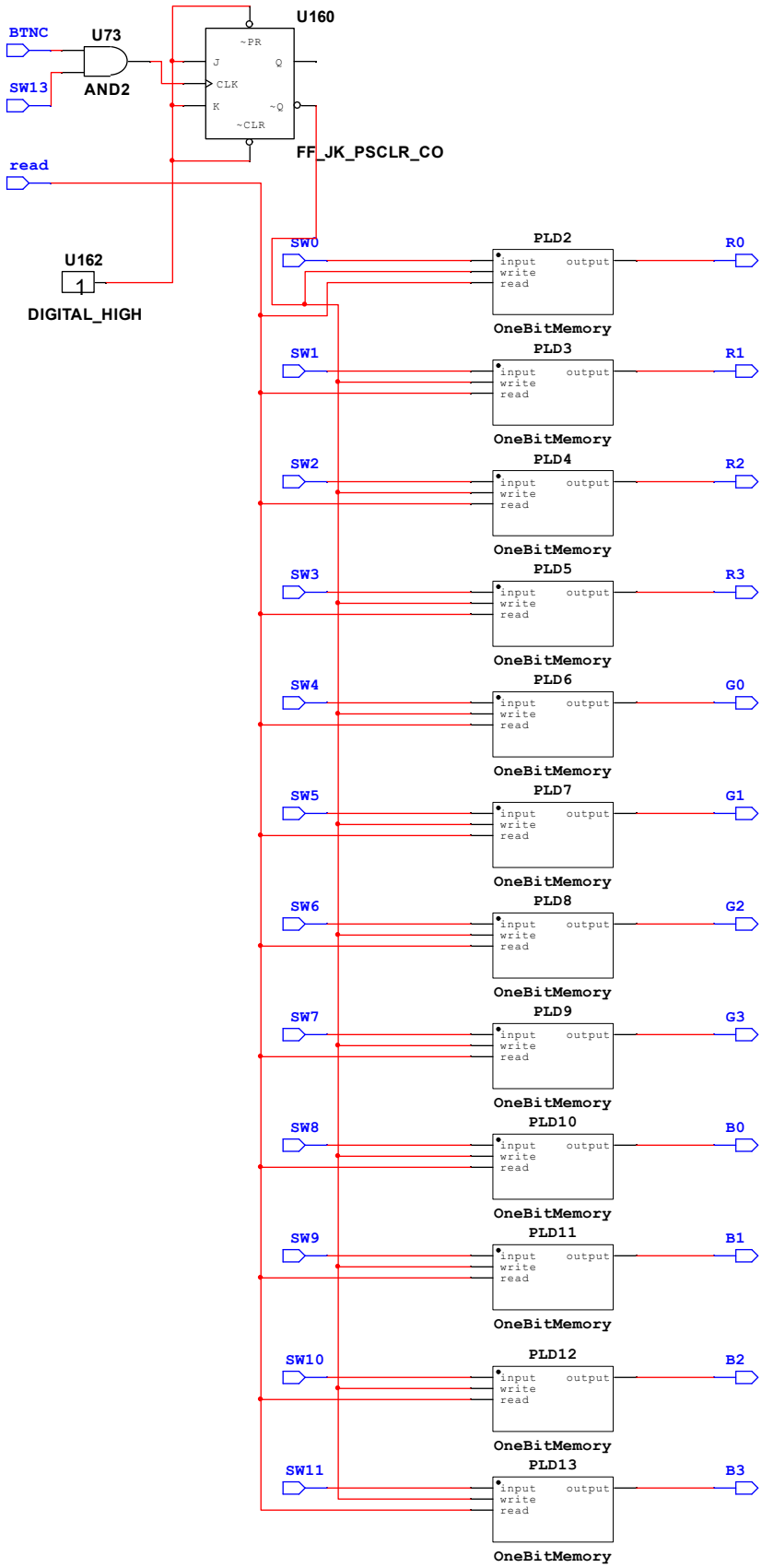
For single colour storage, the RGB signals supplied by SW0-SW11 must be stored by pressing BTNC while SW13 or SW14 is high.

The following schematic shows my single-colour storage about SW13, which consists of 12 1-bit registers, one JK flip-flop and one AND gate with a button input, 12 switch inputs and a read input as well as 12 colour output.

For the JK flip-flop in this circuit, I used the BTNC input and the SW13 input as inputs to a two-input-and-gate, used the output as a clock for a JK flip-flop and input a high level to both the J input and the K input of the JK flip-flop. Since this JK flip-flop is a rising-edge trigger, pressing BTNC when SW13 is high flips the output signal at the output of the flip-flop, which is originally high for Q outputs and low after the flip-flop, and is low for non-Q outputs and high after the flip-flop.

For the 12 1-bit registers, connect SW0-SW11 to the signal input of each register in turn, connect the non-Q output of the above flip-flop to the WRITE input, connect the READ end of each register to an external circuit, and use the 12-bit RGB signal's as the output. Thus, when BTNC is pressed, the register starts to enter the write state, writes the Switch signal inputted from the input into the memory, and outputs the stored RGB signal according to the reading demand of the external circuit.

This circuit is a colour storage circuit regarding SW13 for storing colour 1, and for SW14 for storing colour 2, it can be done by simply making a copy of this circuit and subsequently replacing SW13 with SW14, so it will not be described here.



P34 Circuit of single-colour storage

2.4.3 Mode selection circuit

For the mode selection section, it can be divided into two parts: SW15 for mode

selection and colour switching.Since this function is implemented directly in the top-level circuit of the colour storage, we base our exposition directly on the top-level circuit.

For the selection of the colour mode, a MUX2_1 can be used. The basic logic of MUX2_1 is shown in the table below.

INPUTS				OUTPUT
~G	~A/B	A	B	Y
H	X	X	X	L
L	L	L	X	L
L	L	H	X	H
L	H	X	L	L
L	H	X	H	H

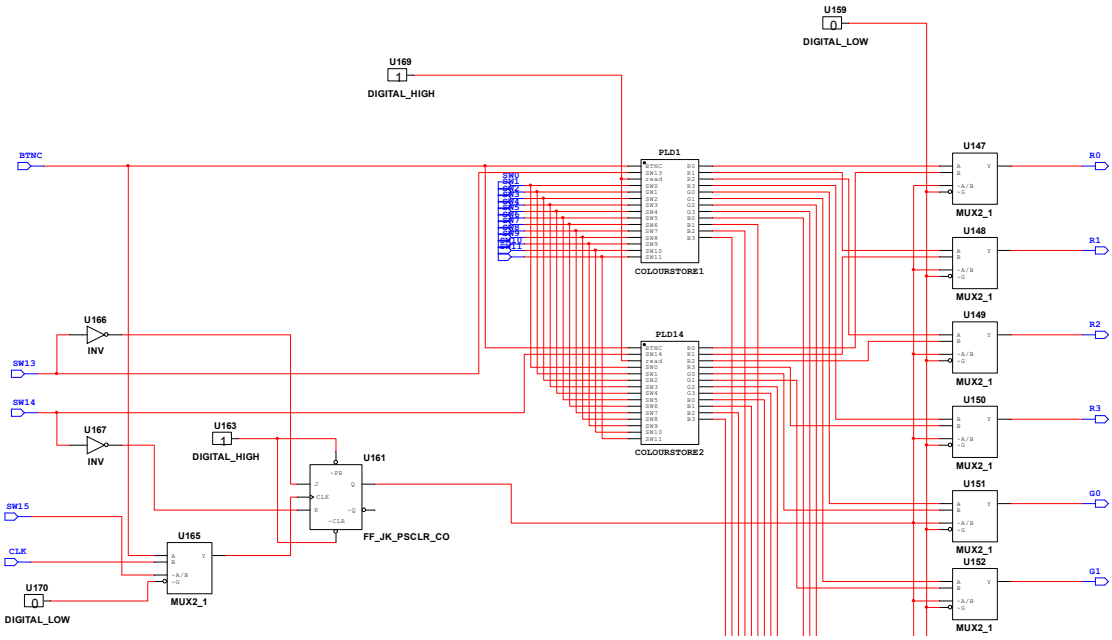
P35 logical of MUX2_1

MUX2_1 operates normally when the G input is low, and outputs the signal from the A input when the A/B input is low, and vice versa for the B input. Therefore, in this circuit, simply connect SW15 directly to A/B. Since SW15 is low when closed and high when open, it conforms to the logic of MUX2_1 selecting the output. For what needs to be input to the A and B inputs, it is not difficult for us to find out according to the experimental requirements that mode1 is to manually press the BTNC to switch the colors while mode2 is to automatically switch the colours in accordance with the 0.5Hz clock signal, so we can connect the BTNC to the A input, connect the 0.5Hz signal from the Clock Divider to the B input, and then subsequently, the output signal The colour mode switching can be achieved by connecting the BTNC to the A input and the 0.5Hz signal of the Clock Divider to the B input, and then using the output signal as the clock input signal of the subsequent colour change circuit.

According to the two colour selection circuits in the previous section, they are connected according to SW0-SW11, SW13, SW14, BTNC respectively one-to-one, and at the same time, the read input is connected directly to the high level. Since the output of the colors in this circuit is determined by BTNC and SW15, the effect of the delay can be eliminated by connecting the read input directly to the high level. Therefore, when we need to store colors, turn on SW13 and COLOURSTORE1 will start to store the selected colors to colour 1; turn on SW14 and it will start to store the selected colors to colour 2; the problem of turning on SW13 and SW14 at the same time will be solved in the subsequent circuits. Subsequently, the two output bits are connected to the 12 MUX2_1 in one-to-one correspondence, which completes the basic construction of the selection circuit, after which the selection conditions are determined according to SW13, SW14, SW15, and BTNC. According to the operation logic of MUX2_1, we should input a low level at A/B terminal when selecting colour 1, and a high level at A/B terminal when selecting colour 2; at the same time, both SW13 and SW14 should be low when the colour is selected, or else it is the state of colour storage. In this circuit, I chose to use a JK flip-flop to press into colour selection. I connected SW13 and SW14 to the J input and K input respectively via a non-gate, connected the output signal of the mode selection circuit to the clock input

of the JK flip-flop, and connected the Q outputs to the A/B terminals of the 12 MUX2_1.

The specific logic is as follows: in the colour storage stage, when SW13 is opened and SW14 is closed, the J input is 0 and the K input is 1. Here, no matter which mode is selected by SW15, the output is low, so the signal output by Q is low, and the colour signal output by the whole circuit is the colour signal of COLOURSTORE1, i.e., Colour 1, and thus Colour 1 is displayed on the screen while Colour 1 is being stored; similarly, when SW14 is opened and SW13 is closed, the signal output by Q of the JK flip-flop is always high, and thus the circuit displays Colour 2 on the screen while Colour 2 is being stored. When SW13 and SW14 are turned on at the same time, J and K are both low, the JK flip-flop is in hold state and the output remains unchanged, so the colors displayed on the screen are still the same as the last signal and will not affect the use. When both SW13 and SW14 are off, J and K are both 1. At this time, the JK flip-flop will flip the output once at each rising edge of the clock. Therefore, in mode1, the input clock is the key, and every time the key is pressed, a rising edge is generated, and the output of the JK flip-flop will flip once, and the output colour will change from colour 1 to colour 2 or 2 to 1. In mode2, the input clock is 0.5, and the input clock is 0.5. In mode2, the input clock is 0.5Hz, and at the rising edge of each cycle of the clock, the output of the JK flip-flop carries out a flip-flop, and the colour of the output is changed once, thus realizing the automatic switching of colors under a certain frequency.



P36 Circuit of color storage

3. Testing

3.1 Colored rectangle

For the testing of Colored rectangle Design, we can judge the correctness of the Basys3 board directly from its experimental results. The testing video is Stage3_rectangle_testing.

Based on the results in the video, we can see that a colored rectangle appears in the upper left corner of the screen and the size of the rectangle matches 480 x 295, so the overall results of the experiment are better.

3.2 Moving rectangle and Coordinate recording

Similarly, judging the correctness of the Basys3 board directly from its experimental results. The testing video is Stage3_moving _testing.

Based on the results in the video, we can see that the rectangle will move up, down, left and right according to BTNU, BTND, BTNL, BTNR, and the coordinates will change normally, at the same time, when moving to the rightmost part of the screen, pressing BTNR will not be able to move right any more, and the number shown on the 7-segment display will be 160 and will not increase any more. Similarly, when moving to the leftmost, topmost and bottom-most parts of the screen, rectangle could not be moved further and the coordinates were stopped at 0,0,185 respectively, so the overall experiment was more successful.

3.3 2 colors storage and 2 coloring modes Design

Similarly, judging the correctness of the Basys3 board directly from its experimental results. The testing video is Stage3_color _testing.

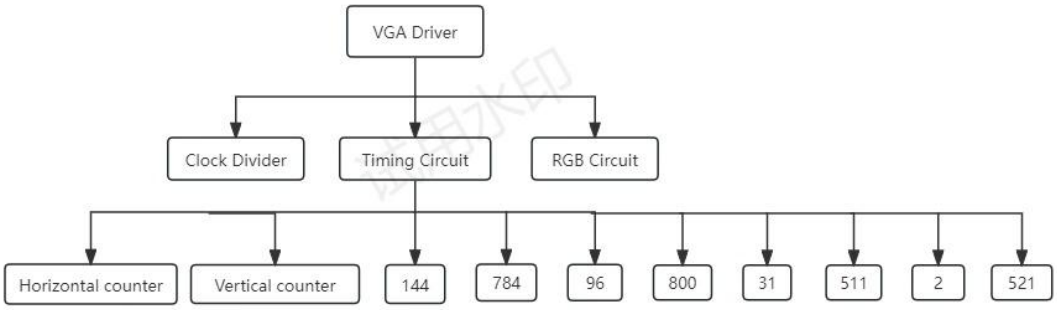
Based on the results in the video, we can see that turn on SW13 and adjust the colour to red, the red can be stored normally to colour 1; Turn on SW14 and adjust the colour to green, the green can be stored normally to colour 2. Turn off SW13, SW14 and SW15, press BTNC frequently, the colours can be switched normally and the switched colours are all stored colours; Turn on SW15, the colours are switched at a frequency of 0.5 Hz. When SW15 is turned on, the colours will be switched at a frequency of 0.5Hz, and the switched colours are all stored colours.

Presentation, discussion, self-reflective statement

1 Discussion of hierarchy

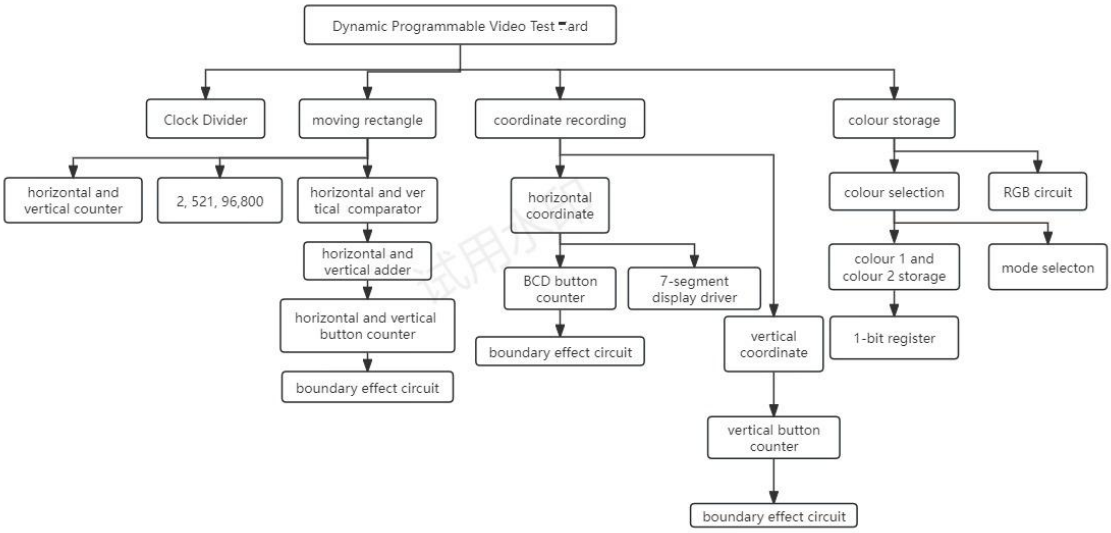
The main task we faced in this project was to design and realize a multi-layered digital system. The concept of hierarchy played a crucial role in this design process. Hierarchies not only help us to manage complexity, but also enable a clearer understanding of the relationships between components and how they work together to achieve overall functionality.

The use of hierarchy is particularly evident in Stage 2 and Stage 3. For Stage2, it has the following hierarchical structure:



P37 hierarchy of stage2

Similarly, Stage3 has the following hierarchical structure:



P38 hierarchy of stage3

My design starts from the topmost level of system architecture and gradually breaks down into sub-modules. These modules are further broken down into smaller units, such as counters, multiplexers, and so on. In this way, we are able to implement and test each part of the system step by step to ensure that each layer works properly and is ultimately integrated into a complete system.

Reasonable hierarchy not only helps to improve the efficiency of the design, but also makes the debugging process more organized. Through the hierarchical division, while making the top-level circuit simpler, the function of each subsystem becomes

clearer and clearer, making the circuit more readable; at the same time, the same subsystems can be used repeatedly and do not need to be rebuilt every time they are used, which greatly improves the efficiency of the circuit construction.

As indicated above, hierarchies make a big complex task into multiple small simple tasks, while making us more organized in the way we think and more able to find solutions to problems.

2 self-reflection

2.1 learning

Firstly, a large number of basic digital logic primitives were learnt. For gates, they perform basic logic operations and determine the flow of signals in a circuit; Flip-Flops are used to store the state of a single bit and are the basic storage unit in a digital system; counters are special logic circuits that record the number of input signals and are used in timing and counting logic; adders are circuits that perform the operation of addition, which is capable of adding two binary numbers and outputting the result; and registers are used to store the state of multiple bits and are used for temporary storage and data transfer.

Also, combinational logic circuits and temporal logic circuits have an important place in this experiment. Combinatorial logic circuits consist of logic gates whose outputs depend only on the state of the current input signal and are independent of the state of previous input signals, including adders, subtractors, etc. Timing logic circuits, on the other hand, take into account the effect of time, and their outputs are not only affected by the current input signals, but are also controlled by the state of the previous input signals and by the system clock signals, including storage elements (e.g., Flip-Flops) and combinatorial logic circuits, which are synchronised and controlled to store and transmit data through the clock signals.

Also VGA is a whole new area for me. VGA provides an analogue video interface for connecting a computer to a monitor for displaying graphics and text.The VGA standard can support a resolution of 640x480 pixels.The scanning process of the monitor is controlled using the HSYNC and the VSYNC.Each pixel can indicate the intensity of the three colour channels, red, green, and blue, using 12 bits.

What's more, I used the Basys3 board for the first time in this experiment and got a basic grasp of FPGA configuration and the use of the VGA port.

2.2 Debugging and Challenges

Since this experiment was a new attempt for me, I basically spent my time building circuits, debugging circuits, finding problems, and solving problems. From simple logic errors to complex synchronization problems. Each failed test forced us to go back to the design stage to check and modify the circuit diagram. The process was tedious and challenging, but it greatly improved our problem-solving skills and deeper understanding of digital circuit design.

For me, the main difficulties arose in the, Stage 2 and Stage 3 phases.

As to Stage2, VGA Driver is a brand new concept, so it is difficult to understand and apply when you first start learning. For the horizontal and vertical timing circuits, it is not possible to correctly understand the meaning and role of the four output signals, so it is difficult to make the correct circuit construction and debugging, and it

is difficult to achieve the correction of the frequency and duty cycle of the waveform. Meanwhile, since different components will produce different gains and effects under the same environment, it is necessary to try and replace them repeatedly during the debugging process until the optimal result is found.

As to the stage3, it is a high volume and more comprehensive experiment.I also encountered a lot of difficulties during the construction and debugging phase of the circuit. For the selection of the counter, since different components have different counting ability and efficiency, it is necessary to find the most suitable counting components through continuous experimentation and debugging of the circuit. Meanwhile, in the process of counting, since it is a key input, the delay has a greater impact on the circuit, and it is necessary to balance the delay problem in the circuit through the use of triggers and other original components. For the design of the colour storage circuit, it is necessary to build the basic register circuit and experiment the function of the circuit by selecting the logic. Therefore, in the process of building the circuit, it is necessary to repeatedly read the requirements of the experiment to understand the logical meaning, and then implement the logic into the circuit. I built the wrong circuit several times due to a misunderstanding of the logic of the topic, which prevented the experiment from yielding the desired results.

3 References and background reading work

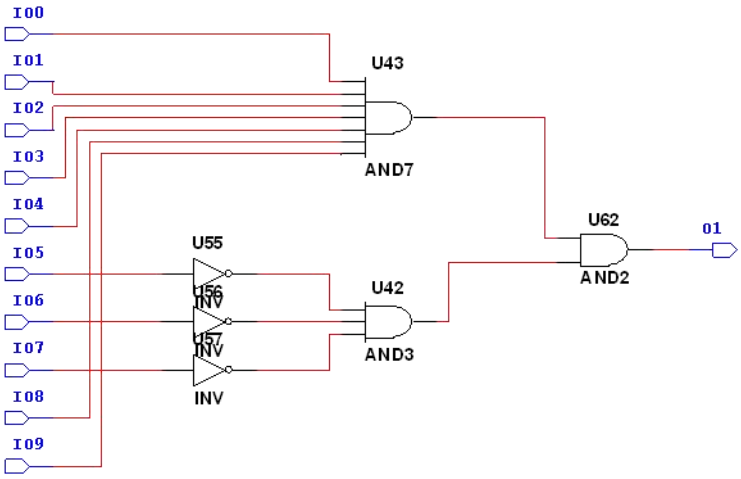
The reference material for this experiment was mainly from the classroom.

[1] basys3_rm.pdf
[2] Basys3-Reference Manual.pdf
[3] CE264_Class2-9
[4] PLD datasheets.pdf
[5] The design of register
Address:www.bilibili.com/video/BV1514y1n7Mk/?spm_id_from=333.337.top_right_bar_window_history.content.click&vd_source=6e83a286c096734659ed3d9bc5e53210

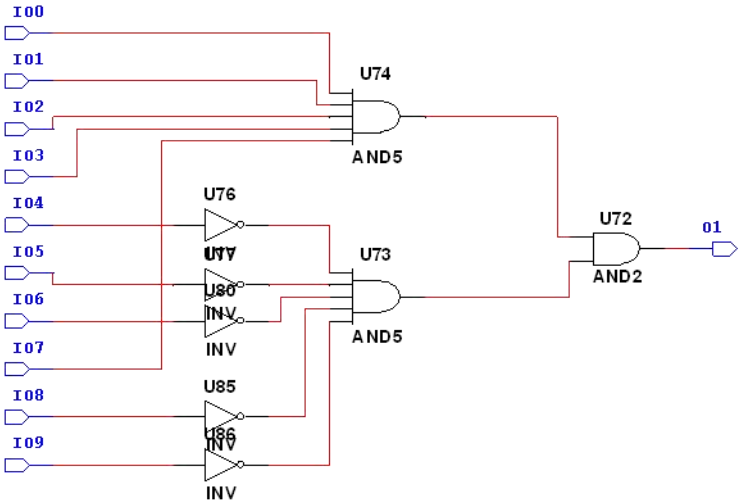
Appendix

For original circuit diagrams not shown in the report, they will be given here.

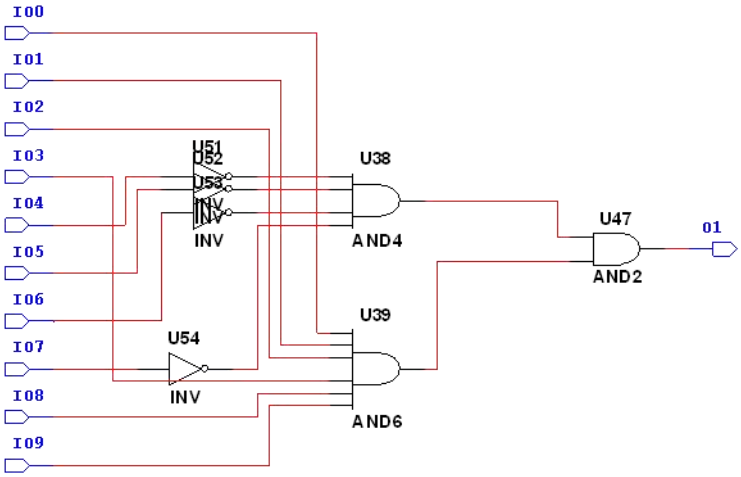
Stage 2:



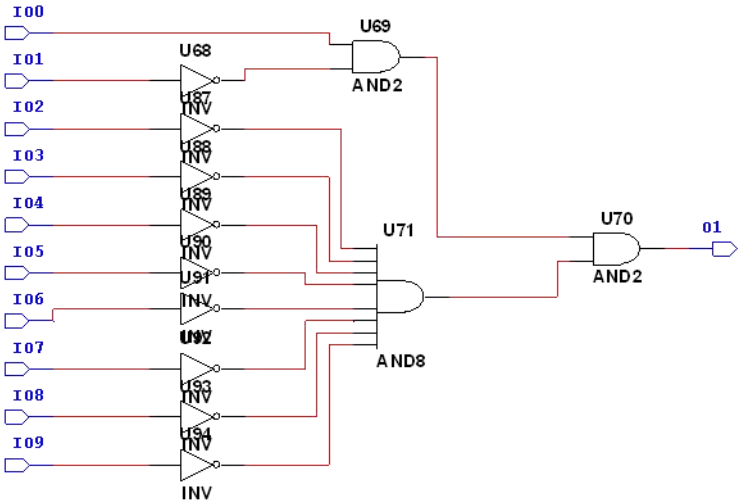
P39 circuit of 800



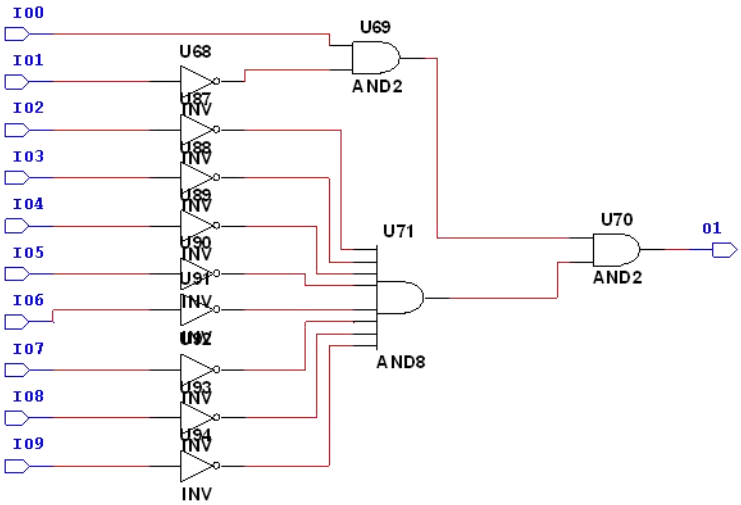
P40 circuit of 144



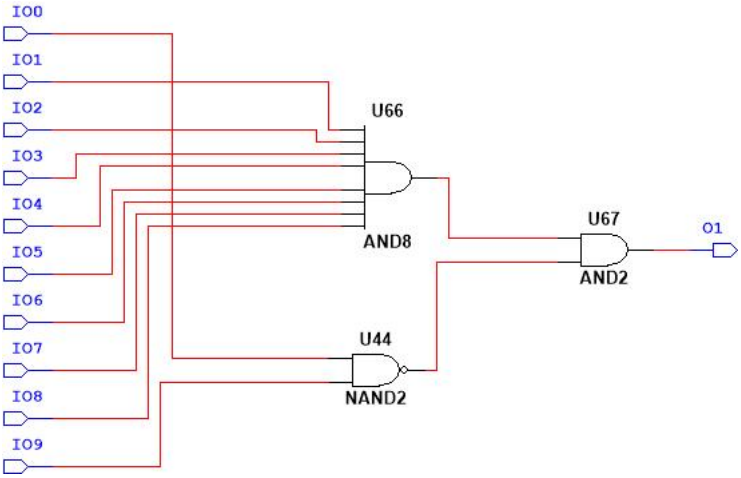
P41 circuit of 784



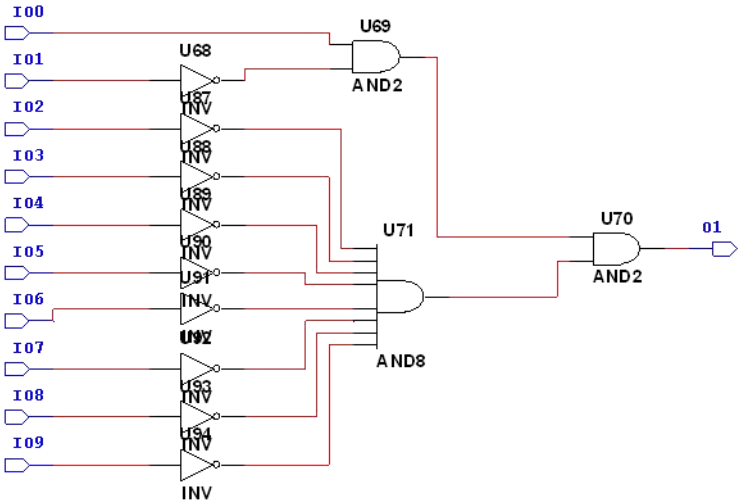
P42 circuit of 2



P43 circuit of 521

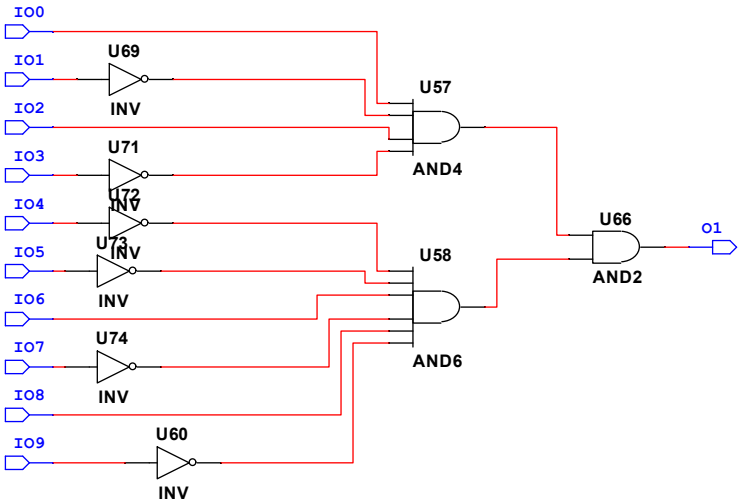


P44 circuit of 511

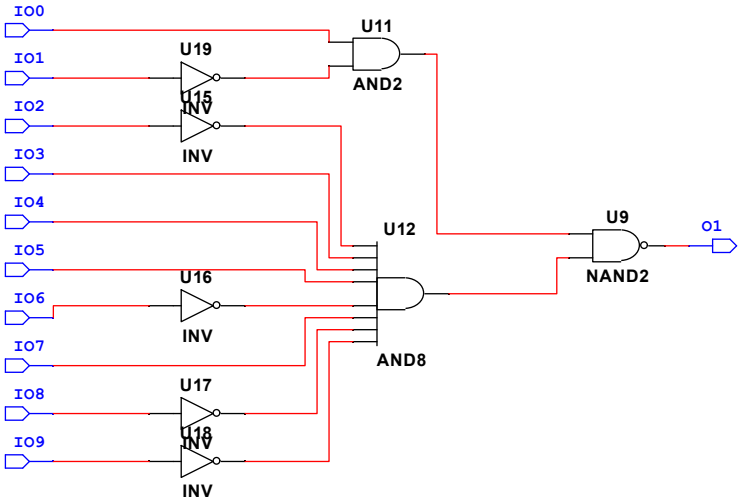


P45 circuit of 31

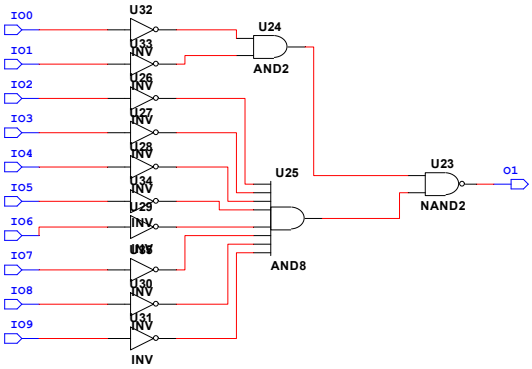
Stage3:



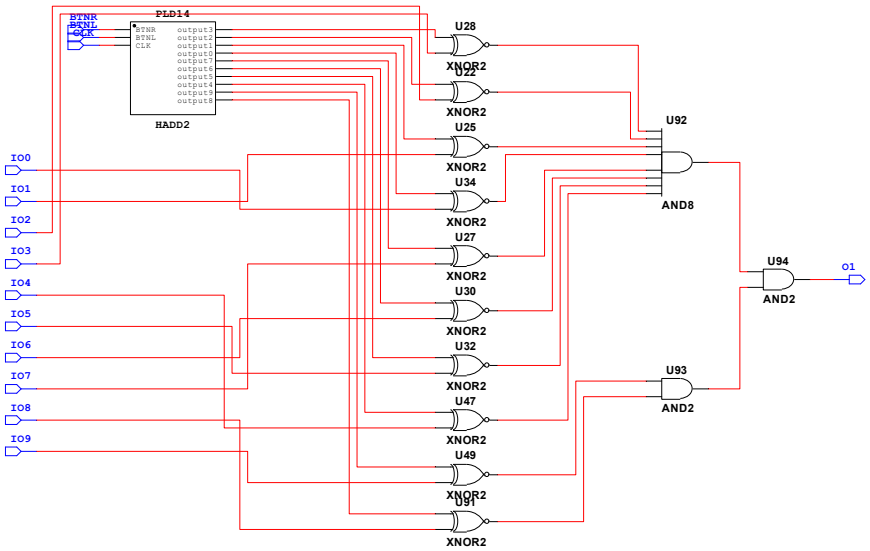
P46 circuit of 326



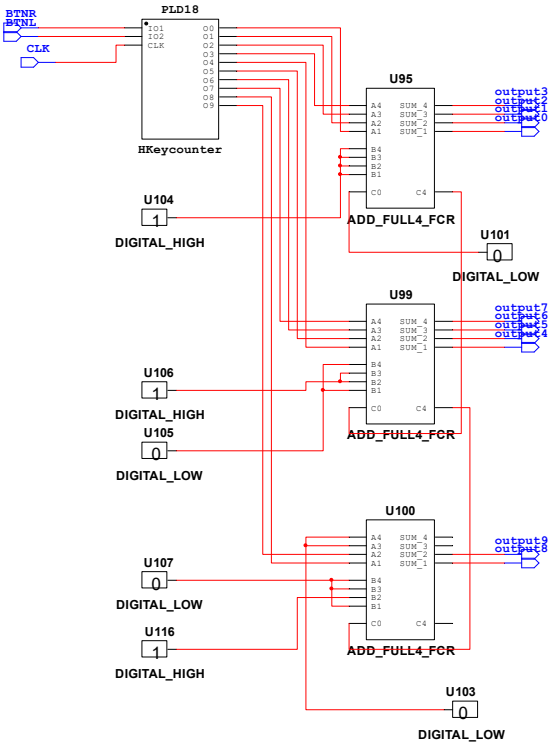
P47 circuit of 185



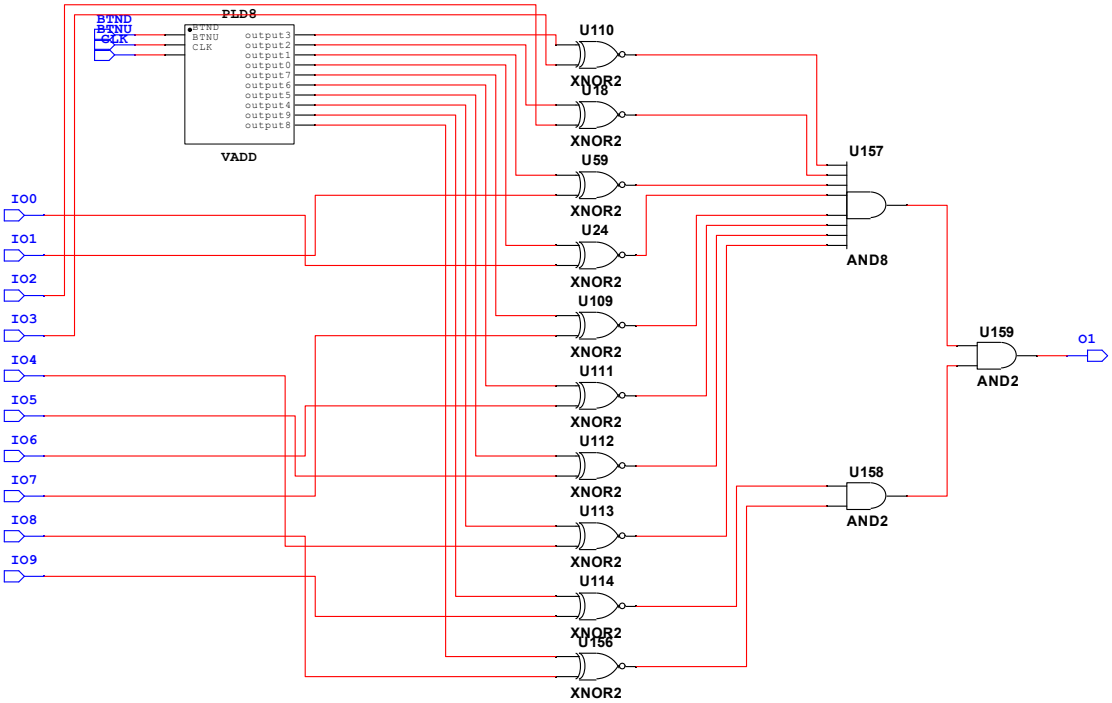
P48 circuit of 0



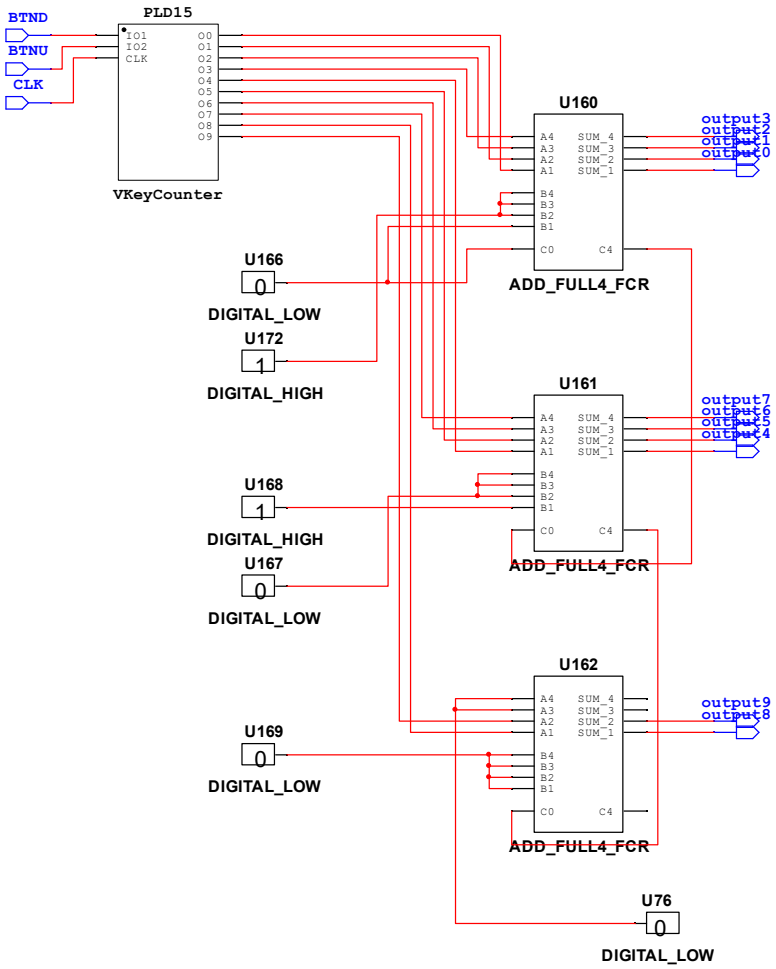
P49 circuit of HCOM2



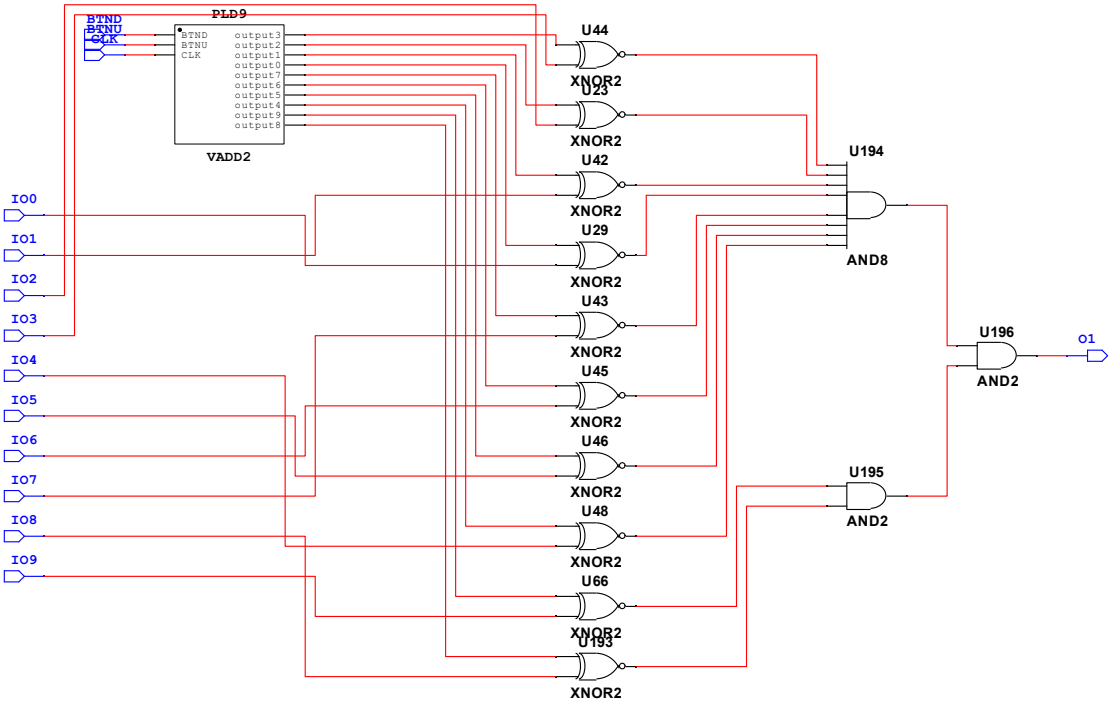
P50 circuit of HADD2



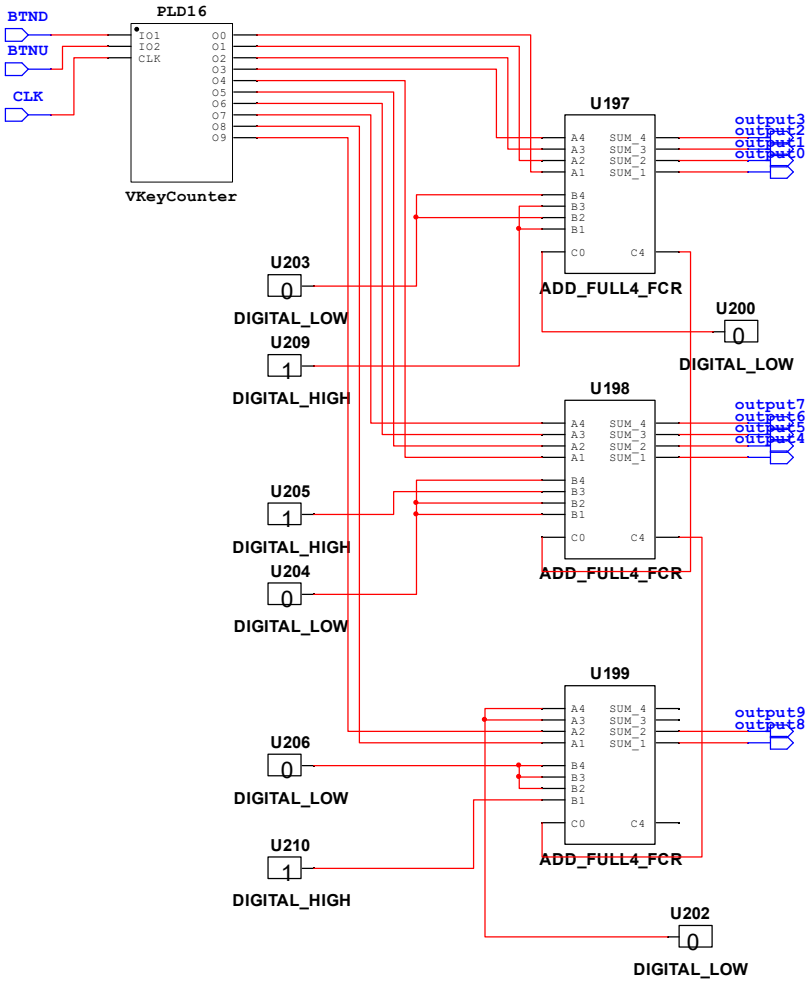
P51 circuit of VCOM1



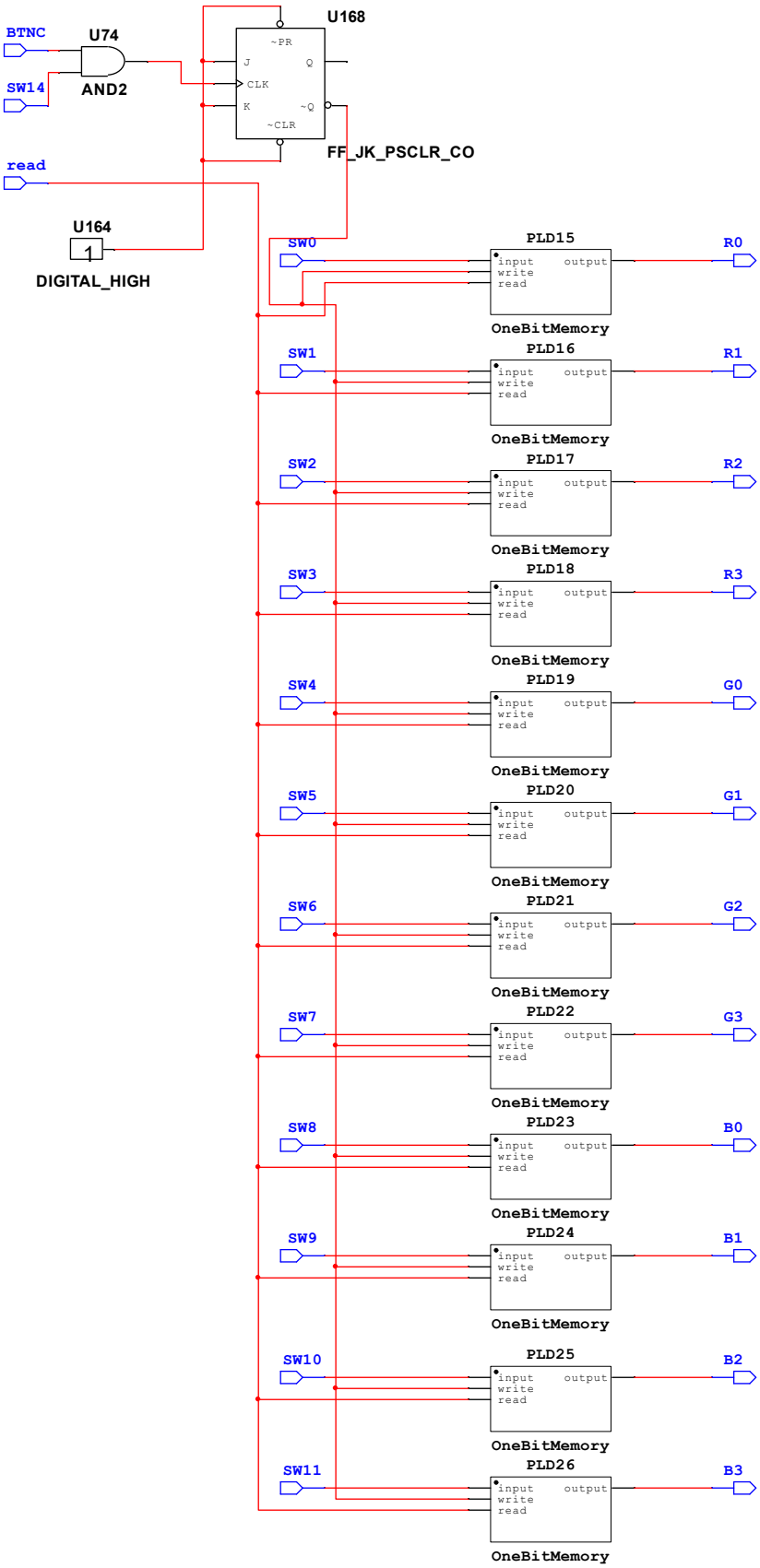
P52 circuit of VADD



P53 circuit of VCOM2



P54 circuit of VADD2



P55 circuit of color storage2